

Intelligent Group Interactive Surveillance and Management System

Final Year Project

2021-2025

A project submitted in partial fulfillment of the degree of

BS in Computer Science



Submitted to

Dr. Ahmad Jalal

Department of Computer Science

Faculty of Computing & Artificial Intelligence (FCAI)

Air University, Islamabad

Type (Nature of project)	☐ Development ☒ R&D			
Area of specialization	☒ WebApp ☐ Mobile App			
	☒ AI based ☐ Embedded System			
FYP ID: 3	F-21-02			
Project Group Members				
Sr.#	Reg. #	Student Name	Email ID	*Signature
(i)	210966	Muhammad Hamdan Azhar	210966@students.au.edu.pk	Hamdan
(ii)	200858	Muhammad Awais	200858@students.au.edu.pk	Awais
(iii)	211958	Muhammad Areeb Shahid	211958@students.au.edu.pk	Areeb

*The candidates confirm that the work submitted is their own and appropriate credit has been given where reference has been made to work of others

Plagiarism Certificate

This is to certify that, I Muhammad Hamdan Azhar Son of Azhar ul Haq group leader of FYP under registration no 210966 at Computer Sciences Department, Air University. I declare that my FYP report is checked by my supervisor.

Date: 24-04-2025 Name of Group Leader: Muhammad Hamdan Azhar Signature: Hamdan

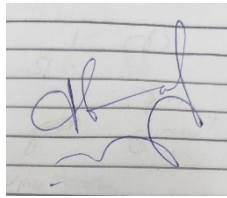
Name of Supervisor: Dr Ahmad Jalal

Co-Supervisor: (Not Allocated)

Designation: Professor

Designation: (N/A)

Signature:

A handwritten signature in blue ink on lined paper. The signature is stylized and appears to be 'Dr. Ahmad Jalal'.

Signature: _____

HoD: Dr. Mehdi Hasan

Signature: _____

Intelligent Group Interactive Surveillance and Management System

Change Record

Author(s)	Version	Date	Notes	Supervisor's Signature
Hamdan	1	24-04-2025	Final change	Ahmad Jalal

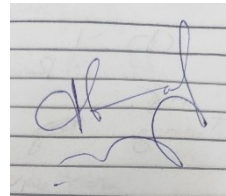
APPROVAL

PROJECT SUPERVISOR

Name: Dr Ahmad Jalal

Date: 24-04-2025

Signature:

A handwritten signature in blue ink on lined paper. The signature is stylized and appears to be 'Dr. Ahmad Jalal'.

PROJECT MANAGER

Date: _____

Signature: _____

CHAIR DEPARTMENT

Date: _____

Signature: _____

Dedication

This work is dedicated to our families

Acknowledgements

We are thankful to our supervisor and our respective families who have helped us along this journey.

Executive Summary

This project focuses on computer vision and addresses the complex problem of human activity surveillance in RGB videos. It is a web-based application that enables users to upload videos of students performing activities in a classroom and receive the corresponding classification of the activity. The application integrates fast and efficient machine learning and deep learning pipelines to perform activity recognition. A subscription feature is available, allowing institution owners to create faculty dashboards for faculty members of their institution. The faculty members can then upload video along with its meta data to store their uploaded videos along with the classification reports. Additionally, an admin panel is provided for effective user and subscription management. Experimental results demonstrate the system's effectiveness, with the machine learning pipeline achieving an accuracy of 87.9% and the deep learning pipeline reaching 88.8% on a combined dataset of our custom-made and academic video samples, highlighting the robustness and practical value of the proposed solution.

Table of Contents

Chapter 1 Introduction & Background	18
1.1. Background	19
1.2. Motivations and Challenges.....	19
1.3. Goals and Objectives	19
1.4. Literature Review/Existing Solutions	20
1.4.1. Handcrafted feature extraction-based classification approaches	21
1.4.2. Deep Learning feature extraction-based classification approaches	21
1.5. Gap Analysis	22
1.6. Proposed Solution	22
1.7. Project Plan	23
1.7.1. Roles & Responsibility Matrix	23
1.7.2. Gantt Chart.....	24
1.8. Report Outline.....	24
Chapter 2 Software Requirement Specifications	25
2.1. Introduction.....	26
2.1.1. Purpose.....	26
2.1.2. Document Conventions.....	26
2.1.3. Intended Audience and Reading Suggestions.....	26
2.1.4. Product Scope	27
2.2. Overall Description	27
2.2.1. Product Perspective.....	27
2.2.2. User Classes and Characteristics	27
2.2.3. Operating Environment.....	28
2.2.4. Design and Implementation Constraints.....	28
2.2.5. Assumptions and Dependencies	28
2.3. External Interface Requirements.....	29
2.3.1. User Interfaces	29
2.3.2. Hardware Interfaces	34
2.3.3. Software Interfaces	34
2.3.4. Communications Interfaces	34
2.4. System Features	34

Chapter 3 Use Case Analysis	37
3.1. Use Case Diagram.....	38
3.2. Use Cases Description	39
Chapter 4 System Design.....	48
4.1. Architecture Diagram.....	50
4.2. Domain Model	51
4.3. Entity Relationship Diagram with data dictionary.....	52
4.4. Class Diagram	53
4.5. Sequence / Collaboration Diagram	54
4.6. Activity Diagram	56
4.7. State Transition Diagram	58
4.8. Component Diagram	59
4.9. Deployment Diagram.....	60
Chapter 5 Implementation.....	61
5.1. Datasets description	62
5.1.1. Students cheating dataset	62
5.1.2. Custom dataset	63
5.2. Segmentation.....	64
5.3. Keypoints detection	65
5.4. Feature Extraction	66
5.4.1. Local ternary pattern features	67
5.4.2. Histogram of Optical Flow (HOF) features	67
5.4.3. Distance Features	68
5.4.4. Velocity features	71
5.5. Classification.....	72
5.5.1. Machine Learning Model.....	72
5.5.2. Deep Learning Model	74
5.6. Experimental Results	77
5.6.1. Confusion matrix	77
5.6.2. Precision, Recall, and F1-Score	78
5.7. Components, Libraries, Web Services and Stubs	78
5.7.1. Components	79
5.8. Deployment environment.....	81
5.9. Tools and Technologies	82

5.10.	Version control.....	83
5.11.	Best practices/coding standards	83
Chapter 6 Business Plan.....		85
6.2	Market Analysis & Strategy.....	86
6.3	Competitive Analysis.....	86
6.4	Products/Services Description	87
6.5	SWOT Analysis	87
Chapter 7 Testing & Evaluation		88
7.1.	Use Case Testing (Test Cases).....	90
7.2.	Equivalence partitioning	120
7.3.	Boundary value analysis	120
7.4.	Data flow testing	120
7.4.1.	Detailed Data Flow Analysis	121
7.5.	Unit testing.....	121
7.6.	Integration testing	121
7.7.	Performance Testing	122
7.8.	Stress Testing	122
Chapter 8 Conclusion & Future Enhancements.....		124
8.1.	Achievements and Improvements	125
8.2.	Critical Review	125
8.3.	Lessons Learnt	125
8.4.	Future Enhancements/Recommendations	126
Appendices.....		127
Appendix A: Information / Promotional Material		128
Reference and Bibliography		129

List of Figures

Figure 1: Gantt Chart	24
Figure 2: Landing Page.....	29
Figure 3: Sign-Up Page.....	29
Figure 4: Admin Analytics Page	30
Figure 5: Admin Dashboard.....	30
Figure 6: Institution Owner Dashboard.....	31
Figure 7: Add/Assign Faculty Page.....	31
Figure 8: Faculty Member Dashboard.....	32
Figure 9: Upload Video Page.....	32
Figure 10: Pricing Page.....	33
Figure 11: Edit Profile Page.....	33
Figure 12: Use Case Diagram	39
Figure 13: Architecture Diagram	50
Figure 14: Domain Model Diagram.....	51
Figure 15: ER-Diagram.....	52
Figure 16: Class Diagram	53
Figure 17: Sequence Diagram for common user and faculty member	54
Figure 18: Sequence Diagram for System Admin	55
Figure 19: Sequence Diagram for Seat limit reached	55
Figure 20: Activity Diagram.....	56
Figure 21: System Admin Activity Diagram	57
Figure 22: State transition diagram.....	58
Figure 23: Component Diagram	59
Figure 24: Deployment Diagram	60
Figure 25: Some samples from the Students Cheating dataset	63
Figure 26: Some samples from our custom dataset	63
Figure 27: Segmentation Results from our experiments.....	65
Figure 28: Keypoints detection and human tracking visualization.....	66
Figure 29: LTP features visualization for different activities	67
Figure 30: HOF features visualization for different activities	68

Figure 31: Intra and inter distance feature visualization for different activities	69
Figure 32: . Intra and inter angle feataures visualization for different activties	70
Figure 33: Velocity heatmap for different activities.....	72
Figure 34: Architecture of Machine Learning pipeline	72
Figure 35: LDA feature optimization	73
Figure 36: Architecture of Multi Class SVM	74
Figure 37: Architecture of Deep Learning pipeline.....	75
Figure 38: LSTM cell Architecture.....	76
Figure 39: Architecture of BiLSTM	77

List of Tables

Table 1: Comparison of Existing Cheating Detection Systems and Proposed Project	20
Table 2: Gaps Identified in Existing Cheating Detection Solutions	22
Table 3: Project Plan with Tasks and Subtasks	23
Table 4: Roles and Responsibilities of Team Members	23
Table 5: Functional Requirement FR-1 — User Authentication & Management	34
Table 6: Functional Requirement FR-2 — Video Upload & Classification.....	34
Table 7: Functional Requirement FR-3 — Subscription Management	35
Table 8: Functional Requirement FR-4 — Institutional Management	35
Table 9: Functional Requirement FR-5 — Faculty and Subject Assignment	35
Table 10: Functional Requirement FR-6 — Automated PDF Report Generation.....	35
Table 11: Functional Requirement FR-7 — Multi-Level Admin Dashboard.....	36
Table 12: Nonfunctional Requirement NFR-1 — Performance	36
Table 13: Nonfunctional Requirement NFR-2 — Security	36
Table 14: Use Case UC-1 — User Sign Up.....	40
Table 15: Use Case UC-2 — User Log In	40
Table 16: Use Case UC-3 — OTP Verification.....	41
Table 17: Use Case UC-4 — Video Processing (Common).....	41
Table 18: Use Case UC-5 — Video Processing (Faculty-Premium).....	42
Table 19: Use Case UC-6 — Admin Subscription Management	42
Table 20: Use Case UC-7 — Add Faculty Member	43
Table 21: Use Case UC-8 — View Storage Analytics	43
Table 22: Use Case UC-9 — View Faculty Dashboard	44
Table 23: Use Case UC-10 — Assign Subject to Faculty	44
Table 24: Use Case UC-11 — Generate PDF Report.....	45
Table 25: Use Case UC-12 — View Stored Video.....	45
Table 26: :Use Case UC-13 — Delete Stored Video (Faculty User)	46
Table 27: Use Case UC-14 — System Admin Removes Institution Admin	46
Table 28: Use Case UC-15 — Logout (All Users).....	47

Table 29: Mappings between keypoints for measuring angles and distances.....	71
Table 30: Confusion matrix for machine learning pipeline	77
Table 31: Confusion matrix for deep learning pipeline	78
Table 32: Measures of Precision, Recall and F1 score for machine learning pipeline	78
Table 33: Measures of Precision, Recall and F1 score for deep learning pipeline	78
Table 34: Test Case TC-1 — Successful User Registration Flow	90
Table 35: Test Case TC - 2 — Successful Admin Registration Flow	91
Table 36: Test Case TC - 3 — Blocking Duplicate Email Registration	92
Table 37: Test Case TC - 4 — Password Strength Validation on Sign-Up	93
Table 38: Test Case TC - 5 — Check Password and Confirm Password match	94
Table 39: Test Case TC - 6 — Rejecting Sign-Up with Missing Required Fields	95
Table 40: Test Case TC - 7 — OTP Verification after Successful Sign-Up	96
Table 41: Test Case TC - 8 — Resend OTP Functionality for Unverified Users	97
Table 42: Test Case TC - 9 — Resend OTP Functionality for Unverified Users	98
Table 43: Test Case TC - 10 — Block OTP verification after 3 failed attempts	99
Table 44: Test Case TC - 11 — Phone Number Format Validation on Sign-Up	100
Table 45: Test Case TC - 12 — Login with Valid Credentials	101
Table 46: Test Case TC - 13 — Reject Login with Invalid Credentials	102
Table 47: Test Case TC - 14 — Reject non video file	103
Table 48: Test Case TC - 15 — Handle Corrupted video upload	104
Table 49: Test Case TC - 16 — Verify concurrent video uploads	105
Table 50: Test Case TC - 17 — Verify common user can upload video without metadata	105
Table 51: Test Case TC - 18 — Verify Faculty member uploads video with metadata	107
Table 52: Test Case TC 19 — Institutional admin creates faculty member	108
Table 53: Test Case TC - 20 — Faculty can delete stored video	109
Table 54: Test Case TC- 21 — Block non-admins from user management	110
Table 55: Test Case TC 22 — Admin Toggles Subscription Status	111
Table 56: Test Case TC 23 — Admin Deletes a User Account	112
Table 57: Test Case TC 24 — Institutional admin views faculty list	113
Table 58: Test Case TC - 25 — Generate PDF Report for Video	114
Table 59: Test Case TC 26 — Faculty member views assigned subjects	115

Table 60: Test Case TC - 27 — Admin views all analytics.....	116
Table 61: Test Case TC - 28 — Institutional Admin Assign subjects to Faculty Member	117
Table 62: Test Case TC - 29 — Video Upload by Faculty Member	118
Table 63: Test Case TC - 30 — Video upload by Common User	119
Table 64: Equivalence Partitioning for Input Validation.....	120
Table 65: Boundary Value Analysis for Input Limits	120
Table 66: Integration Testing Results	122
Table 67: Performance testing results.....	122

List of Abbreviations

ML	Machine Learning
DL	Deep Learning

Chapter 1 Introduction & Background

Chapter 1: Introduction

1.1. Background

Examinations are critical in assessing students' academic performance, yet the integrity of this process is often compromised by various forms of cheating. Traditionally, invigilators are responsible for identifying suspicious behavior; however, human limitations such as fatigue, bias, or distraction can reduce the accuracy and effectiveness of manual surveillance. With advancements in artificial intelligence, particularly in computer vision, it has become possible to detect and classify human activities automatically. This has paved the way for systems that ensure fair examination environments through automated monitoring.

Intelligent Group Interactive Surveillance and Management System is designed to address this challenge by offering an intelligent web-based solution that leverages ML and DL models to analyze student behavior from RGB video footage. It classifies group interactions into categories such as *doing own work*, *passing paper*, and *looking at others' work*.

1.2. Motivations and Challenges

The increasing concern over academic dishonesty in classrooms has motivated this project. Manual invigilation is insufficient for large classrooms and can lead to overlooked incidents. Moreover, no unified, intelligent, and affordable system currently exists to automatically detect group-based cheating behaviors using video analysis.

Challenges include:

- Collecting high-quality classroom footage.
- Training accurate models that generalize across different classroom settings.
- Designing user-friendly web interfaces.
- Role-based feature access and secure data management.

1.3. Goals and Objectives

The goal of project is to develop a robust, intelligent surveillance system that can:

- Analyze classroom videos to detect cheating behaviors.
- Supports institutional hierarchies (system admin → institutional admin → faculty).

- Delivers actionable insights via PDF reports
- Offer admin-level access for managing users and subscriptions.

Objectives:

- Develop Machine Learning (ML) and Deep Learning (DL) pipelines for activity classification.
- Develop a secure NextJs Django-based web application.
- Integrate cloud storage options based on user subscription.
- Deliver accurate classification results for proper analysis.

1.4. Literature Review/Existing Solutions

Some systems have proposed methods for student activity classification whose weaknesses and our proposed project solution are given in the following table:

Table 1: Comparison of Existing Cheating Detection Systems and Proposed Project Improvements

System	Weakness	Proposed Project Solution
[1]	Training data is limited. So the predictions of the proposed system are not reliable.	Our system is designed to detect cheating activities involving groups of students having interactions. With a larger and more diverse training dataset, the system's predictions are more reliable.
[2]	Built to classify activities from blurry CCTV footage from a top angle, not very suitable to regular RGB videos recorded from a classroom perspective.	Our system is specifically built to classify activities in high quality RGB videos recorded from a classroom perspective.

However, no existing solution comprehensively classifies group cheating behavior through automated video processing using both ML and DL models in a web-based SaaS model — this is what distinguishes our system.

Many works have been conducted in the broader domain of group activity recognition, beyond student activity classification involving two-person interactions, where additional classes of activities are also considered. This section gives an overview of human interaction classification by handcrafted feature extraction-based approaches and deep learning feature extraction-based approaches.

1.4.1. Handcrafted feature extraction-based classification approaches

These approaches typically began with preprocessing steps—such as histogram equalization and gamma correction to reduce noise. Then segmentation techniques, such as Watershed, Graph Cuts, and Active Contours, were commonly used for human segmentation but were highly sensitive to image quality. For keypoints detection, techniques such as skeletonization of silhouettes and contour-based joint estimation, have been applied to estimate joint positions.

Then a variety of features are extracted. For segmentation, this included features like SIFT, HOG, Motion History Image, Bag of Words-based descriptors and Motion Boundary Histograms while for keypoints, this included keypoint angles, distances and pairwise keypoint displacements, to capture the spatial and temporal dynamics of the interaction. Then these features are optionally optimized using techniques like Linear Discriminant Analysis or t-SNE before being fed into machine learning classifiers such as Support Vector Machines, Random Forests and k-Nearest Neighbors for final classification.

Some more recent works also incorporated depth information alongside RGB data. For example, in [3], both gray scale and depth silhouette features along with point-based features were extracted before cross entropy optimization and classification using a Maximum Entropy Markov Model.

1.4.2. Deep Learning feature extraction-based classification approaches

Many approaches have been explored to extract both spatial and temporal features using deep learning models for human interaction classification. This often involve finetuning Convolutional Neural Network (CNN) architectures to capture spatial features from individual frames, along with 3D CNNs or Recurrent Neural Networks to model temporal dynamics across sequences. Segmentation-based deep learning models such as Mask R-CNN and U-Net, trained on diverse RGB datasets, have also gained popularity over time due to their ability to operate across varying video resolutions. For example, [4] used a DenseNet encoder with a custom decoder for semantic segmentation, refining inputs by suppressing irrelevant regions before passing them into an EfficientNet-BiLSTM for classification.

Researchers have also explored multi-stream architectures, where each stream processes a different modality—such as RGB frames, optical flow, or keypoint coordinates—and combines

the information using fusion techniques. This modular design allows each stream to specialize in a particular aspect of the interaction, improving overall performance.

More recently, there has been increasing focus on systems that explicitly model spatial and temporal relationships between interacting individuals. With the advent of advanced keypoint detection models like OpenPose [5], modeling interactions through body joint movements has become more feasible. Works such as [6] and [7] have leveraged Interactional Relational Networks and Graph Convolutional Networks (GCNs) to effectively represent both intra- and inter-person relationships, offering promising results in complex interaction scenarios.

1.5. Gap Analysis

Table 2: Gaps Identified in Existing Cheating Detection Solutions

Existing Solutions	Gaps Identified
Require live invigilation	Cannot classify activities in offline classroom recordings
Focus on individual actions	Miss contextual group activities like “passing papers”
Commercial tools are expensive	Lack of accessible, open-source or student-built options

Project fills this gap by providing a classroom-focused group interaction classification system using recorded footage.

1.6. Proposed Solution

Project proposes an AI-driven web application that accepts video input and classifies group activity using ML and DL pipelines. ML and DL models are trained using both custom and academic datasets. The platform:

- Offers role-based access:
 - System Admin: Manages institutions/subscriptions.
 - Institutional Admin: Oversees faculty (10 seats/institution).
 - Faculty: Uploads videos, receives PDF reports.
 - Common Users: Basic ML analysis.
- Admin panel allows user management and subscription control.
- Classification result is returned in the form of pdf to the user.

1.7. Project Plan

Table 3: Project Plan with Tasks and Subtasks

Task	Subtasks
Requirement Analysis	Literature Review, Interviews
Model Development	Dataset preparation, ML/DL model training
Backend Development	Django setup, API design
Frontend Development	NestJs setup, Login/Signup UI, Video Upload UI
Integration	ML/DL + Web App, MongoDB + GCP
Testing & Validation	Unit testing, model testing
Documentation	Report writing, diagrams

1.7.1. Roles & Responsibility Matrix

Table 4: Roles and Responsibilities of Team Members

Role	Person Responsible
Machine Learning and Deep Learning Pipelines	Muhammad Hamdan Azhar
Partial Frontend and Dataset Preparation	Muhammad Areeb Shahid
Partial Frontend & Backend	Muhammad Awais

1.7.2. Gantt Chart

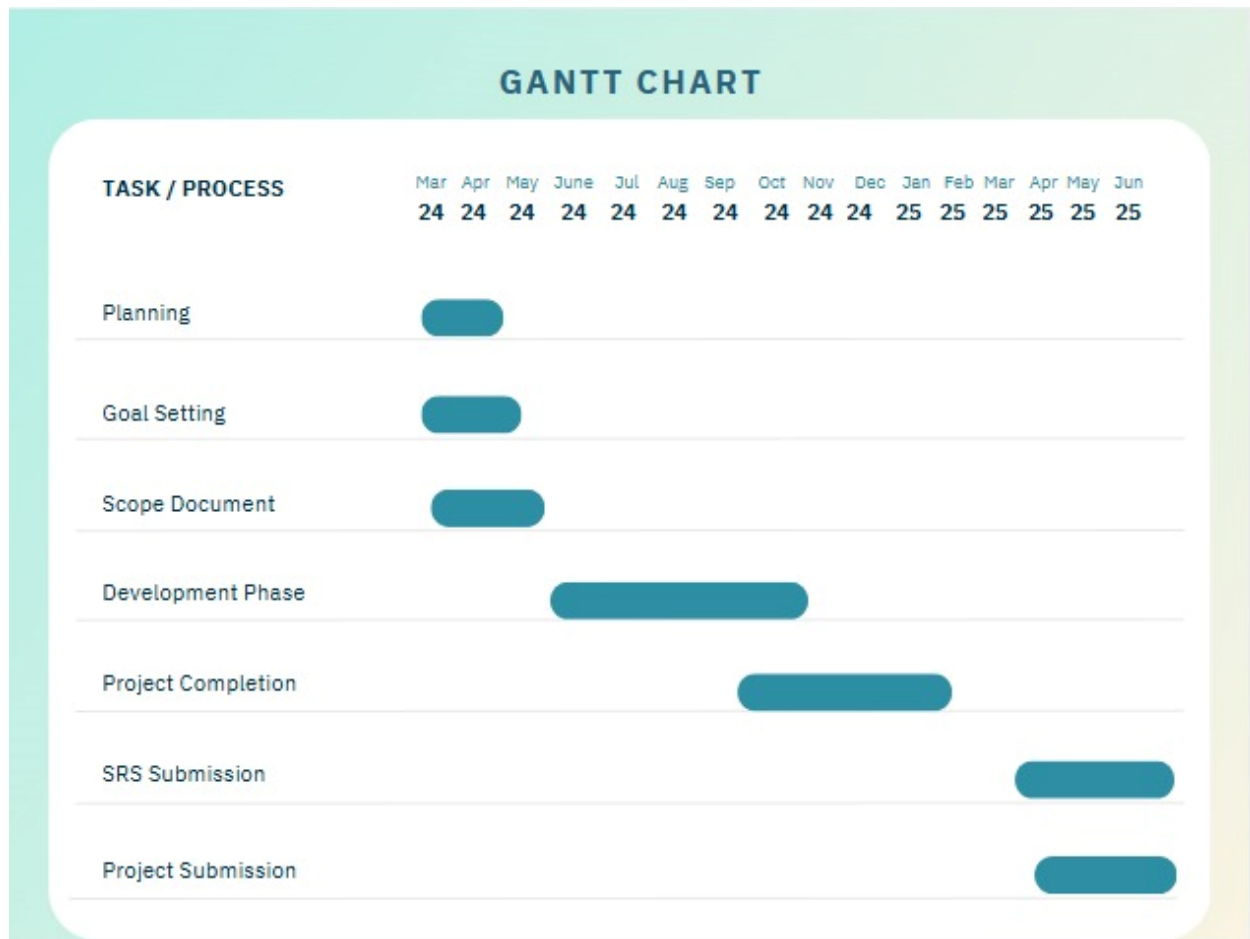


Figure 1: Gantt Chart of Intelligent Group Interactive Surveillance and Management System

1.8. Report Outline

This report begins with an introduction covering background, goals, and a gap analysis of existing solutions. It then details the software requirements and system features of our project in Chapter 2. Chapter 3 analyzes use cases, while Chapter 4 presents system design diagrams. The ML and DL pipelines implementation along with the project's deployment, components are discussed in detail in Chapter 5, followed by a business plan in Chapter 6. Testing strategies are explored in Chapter 7, and the report concludes with achievements, reviews, and future recommendations in Chapter 8.

Chapter 2

Software Requirement Specifications

Chapter 2: Software Requirement Specifications

2.1. Introduction

This chapter outlines the software requirements for the proposed web-based application. This document covers the technical and functional specifications required to build the system, including the software architecture, user interface, system features, and external dependencies, providing a detailed guide for development, testing, and deployment.

2.1.1. Purpose

This document specifies the requirements for our proposed web-based application that classifies cheating behaviors among students in classroom environments using ML and DL pipelines. The system will help educators maintain academic integrity by automatically identifying suspicious activities during exams.

1. Detects cheating behaviors via ML/DL pipelines.
2. Supports multi-tier administration (system admin → institutional admin → faculty).
3. Generates PDF reports with actionable insights.

2.1.2. Document Conventions

This document follows standard formatting and typographical conventions to maintain clarity and consistency. The conventions used are as follows:

- **Font Style & Size:** All text is written in **Times New Roman, 12 pt**, with headings in **14 pt, Bold**.
- **Line Spacing:** The document uses **1.5 line spacing** for better readability.
- **Text Alignment:** All paragraphs are **justified** to provide a clean and organized appearance.
- **Section Headings:** Major section headings are written in **Bold** and have a larger font size for emphasis.
- **Numbering:** Sections and sub-sections are numbered hierarchically (e.g., 1, 1.1, 1.2, 2, 2.1, etc.).

2.1.3. Intended Audience and Reading Suggestions

This document is intended for the following audience:

- **Developers:** To use as a guide for system implementation.

- **System Admin:** Toggle and remove the subscription of Institutional Admin.
- **Institutional Admin:** Manage faculty and subject assignments
- **Project Managers:** To monitor project deliverables and milestones.
- **Testers:** To design test cases for functional and nonfunctional requirements.
- **End Users:** To understand the system's functionality and capabilities.

2.1.4. Product Scope

The system will:

- Process classroom video footage and classify student activities into three categories: *doing own work, passing paper, looking at other's work.*
- Provide tiered access based on subscription status storing videos on the cloud.
- Offer administrative controls for user management.

2.2. Overall Description

2.2.1. Product Perspective

The system integrates multiple components into a cohesive workflow:

- **Frontend (NextJs):** Provides a user-friendly interface for uploading videos, selecting models, and viewing classifications.
- **Backend (Django):** Manages business logic, user authentication, and communication with other components.
- **Database (MongoDB):** Stores user details and video metadata for subscribed users.
- **Storage (Google Cloud):** Stores videos uploaded by subscribed users and their processed results.

This product is a self-contained system designed to operate independently, with external dependencies limited to cloud-based services like Google Cloud and MongoDB Atlas.

2.2.2. User Classes and Characteristics

User Classes

System Admin

- **Capabilities:** Toggle institutional subscriptions.
- **Constraints:** No direct faculty access

Institutional Admin

- **Capabilities:** Add/remove faculty, assign subjects

- **Constraints:** Max 10 faculty/institution

Faculty User

- **Capabilities:** Upload videos, view PDF reports
- **Constraints:** DL model requires premium

Common User

- **Capabilities:** Basic ML analysis
- **Constraints:** No storage

2.2.3. Operating Environment

- **Frontend:** Built using NextJs, accessible on modern web browsers (Chrome, Firefox, Edge).
- **Backend:** Runs on Django, hosted on cloud servers.
- **Database:** MongoDB Atlas for secure, scalable storage of user and video metadata.
- **Storage:** Google Cloud Storage for reliable and scalable video file storage.

2.2.4. Design and Implementation Constraints

- **Hardware:** High-performance cloud-based GPU required for deep learning inference.
- **Software Stack:** Restricted to NextJs, Django, Django REST framework, and AI libraries for model development.
- **Security:** All data transmissions must use HTTPS and token-based authentication.

2.2.5. Assumptions and Dependencies

- Google Cloud Storage and MongoDB Atlas will be available throughout the system's lifecycle.
- AI models are in working conditions and give proper accuracy, to handle processing properly.
- Users will access the system using devices with stable internet connections
- Stable internet connection for communication between frontend and backend services.
- Secure communication protocols (HTTPS) for data privacy and protection.

2.3. External Interface Requirements

2.3.1. User Interfaces

The following are User Interface screens of our system:

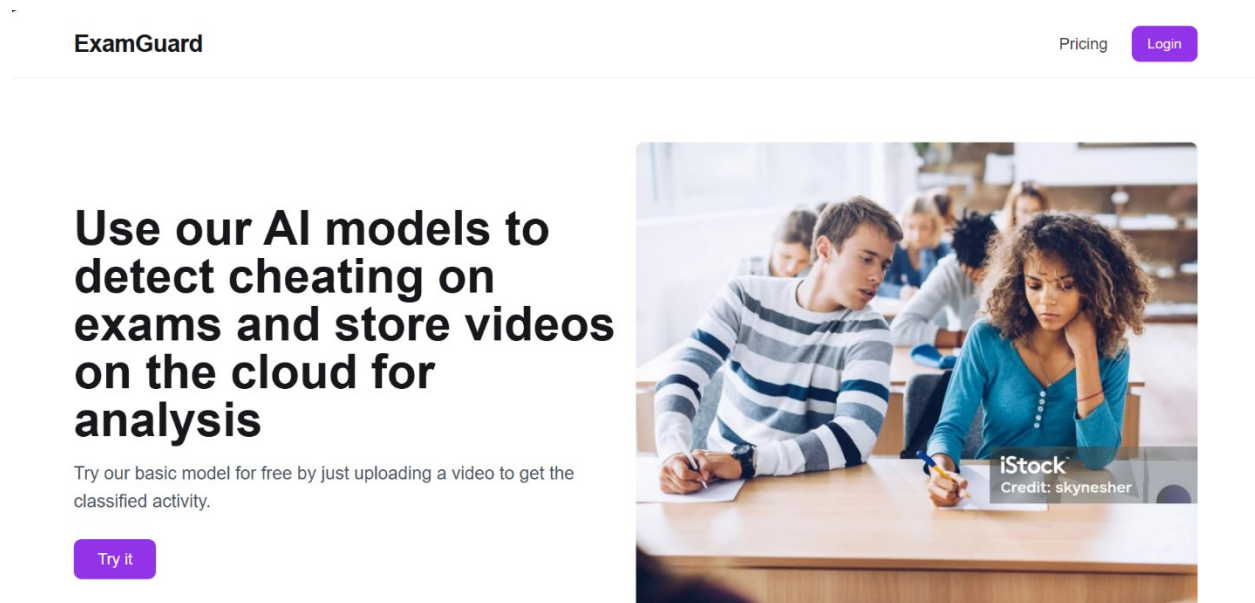


Figure 2: Landing Page of Intelligent Group Interactive Surveillance and Management system

Create an account

Enter your username

Email address

Enter your institution name

Karachi

Password

Confirm password

Type your phone number

User

☐ I agree to the Terms of Service and Privacy Policy

Figure 3: Sign-Up Page of Intelligent Group Interactive Surveillance and Management system



Figure 4: Admin Analytics Page of Intelligent Group Interactive Surveillance and Management system

ExamGuard

Analytics Requests hamdan_admin • Admin Logout

Search members All Locations

Name	Email	Joined	Phone	Institution	Location	Subscription	Action
Areeb Shahid	211958@students.au.edu.pk	2024-10-30	03412841693	Ali public school	Karachi	Inactive	Remove
moizk678	morty2108977@gmail.com	2025-05-25	03392003651	Irfan university	Lahore	Inactive	Remove
hamdan	ali@gmail.com	2025-01-22	03341111318	Sadiq Public School	Karachi	Active	Remove
Aoun	aounmalik05@gmail.com		03061527877	Aoun college	Islamabad	Inactive	Remove
Areeb	mianareeb007@gmail.com	2025-05-27	03090143616	Khawaja Farid	Karachi	Active	Remove
subhan	subhanazhar098@gmail.com	2025-05-27	03341111219	Quetta university	Quetta	Active	Remove
M Ali	200858@students.au.edu.pk	2025-05-27	03001231231	Ahmed School	Karachi	Inactive	Remove

Figure 5: Admin Dashboard of Intelligent Group Interactive Surveillance and Management system

ExamGuard

Assign/Add faculty hamdan • Institution Owner Logout

Remaining Seats: 8

Name	Email	Phone	Password	Program	Semester	Subject	Timing	Actions
Hamdan_faculty	210966@students.au.edu.pk	03341111311	123456aA	BSCS	4	Human Resource	23:00	Remove Subject Delete User
				BS EE	4	English	12:00	Remove Subject
Areeb Shahid	learneras25@gmail.com	03261021195	123456aA	CS	8th	Social Service	09:30	Remove Subject Delete User

Figure 6: Institution Owner Dashboard of Intelligent Group Interactive Surveillance and Management system

Add New Faculty Member

Create Faculty

Assign Subject

Assign Subject

Figure 7: Add/Assign Faculty Page of Intelligent Group Interactive Surveillance and Management system

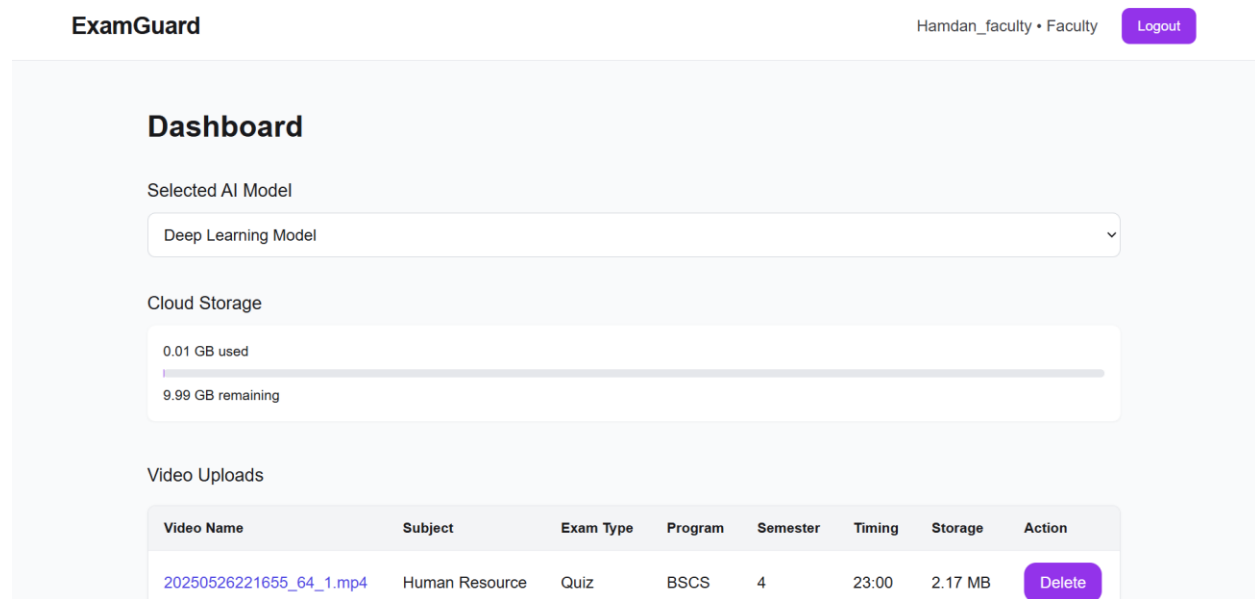


Figure 8: Faculty Member Dashboard of Intelligent Group Interactive Surveillance and Management system

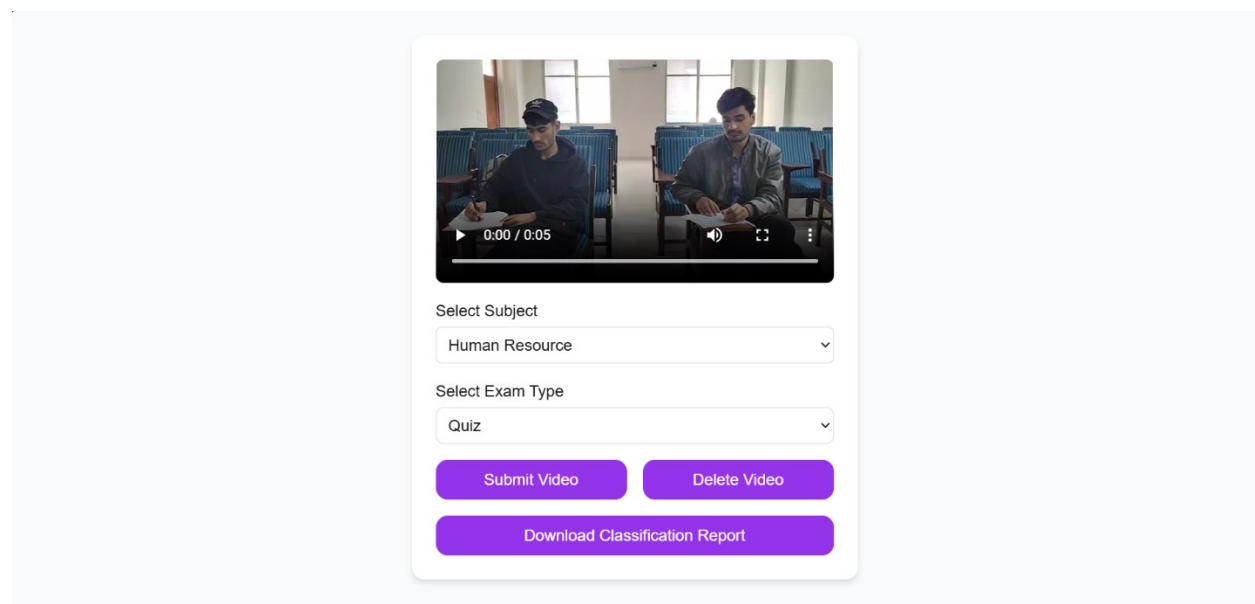


Figure 9: Upload Video Page of Intelligent Group Interactive Surveillance and Management system

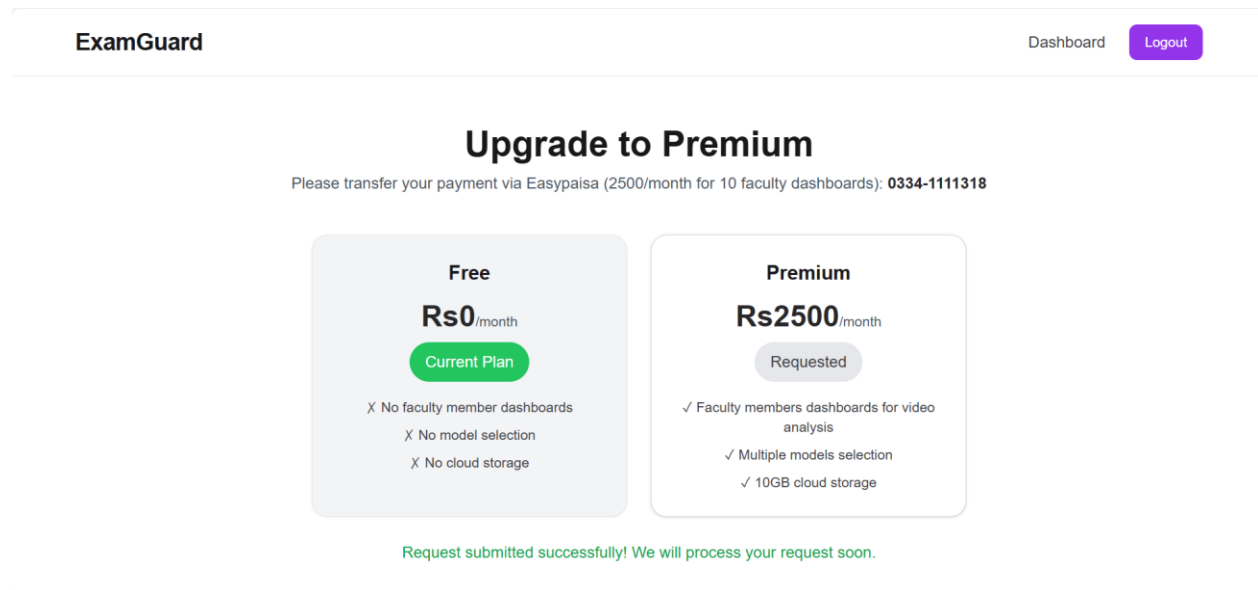


Figure 10: Pricing Page of Intelligent Group Interactive Surveillance and Management system

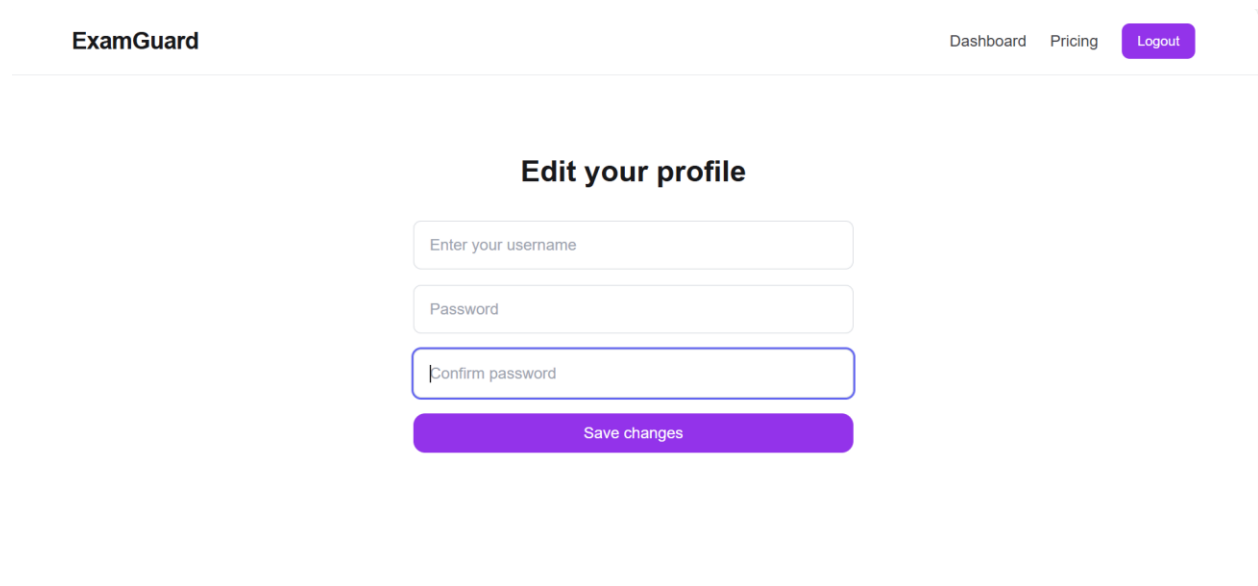


Figure 11: Edit Profile Page of Intelligent Group Interactive Surveillance and Management system

2.3.2. Hardware Interfaces

- Minimum client requirements: 4GB RAM, modern browser
- Server requirements: 8GB RAM, Telsa T4 GPU for ML and DL models processing during training and inference

2.3.3. Software Interfaces

- Google Cloud Storage API
- MongoDB Atlas

2.3.4. Communications Interfaces

- REST API endpoints
- HTTPS protocol for all communications

2.4. System Features

Table 5: Functional Requirement FR-1 — User Authentication & Management

Identifier	FR-1
Title	User Authentication & Management
Requirement	The system will allow users to register via email with OTP verification and log in using JWT-based authentication.
Source	Project Proposal
Rationale	Ensures secure access and prevents unauthorized use
Business Rule	All users must verify their email before accessing features.
Dependencies	MongoDB (user storage), OTP emails
Priority	High

Table 6: Functional Requirement FR-2 — Video Upload & Classification

Identifier	FR-2
Title	Video Upload & Classification
Requirement	The system will accept video uploads (MP4/MOV), process them via ML/DL pipelines, and return classifications ("passing paper", "looking at others", "doing own work").
Source	Project Proposal
Rationale	Core functionality for cheating detection.
Business Rule	Subscribed users get DL model access; others only ML.
Dependencies	Google Cloud Storage (subscribers), Ultralytics (keypoints detection), MASK R-CNN (segmentation), OpenCV (feature extraction), scikit-learn/pytorch (classification)
Priority	Very High

Table 7: Functional Requirement FR-3 — Subscription Management

Identifier	FR-3
Title	Subscription Management
Requirement	The system shall restrict DL model and GCS storage to paid subscribers. Admins shall manually override subscriptions.
Source	Project Proposal
Rationale	Ensure premium features are only available to paid users
Business Rule	Paid users can access DL models and GCS storage; admins can manually override subscription restrictions.
Dependencies	Payment
Priority	Medium

Table 8: Functional Requirement FR-4 — Institutional Management

Identifier	FR-4
Title	Institutional Management
Requirement	System shall allow System Admin to: 1. Create/delete institutions 2. Set storage quotas 3. Toggle subscription status for institutions
Source	Project Proposal
Rationale	Centralized control for multi-school deployment
Business Rule	Paid users can access DL models and GCS storage; admins can manually override subscription restrictions.
Dependencies	MongoDB (institution schema), Google Cloud Storage API
Priority	High

Table 9: Functional Requirement FR-5 — Faculty and Subject Assignment

Identifier	FR-5
Title	Faculty and Subject Assignment
Requirement	Institutional Admin shall: 1. Add/remove faculty members (max 10/institution) 2. Assign subjects with timing slots 3. Resolve scheduling conflicts automatically
Source	Project Proposal
Rationale	Ensures proper academic workload distribution
Business Rule	1. Subject timings in HH:MM-HH:MM format 2. No overlapping slots per faculty
Dependencies	MongoDB (faculty_subject collection)
Priority	High

Table 10: Functional Requirement FR-6 — Automated PDF Report Generation

Identifier	FR-6
Title	Automated PDF Report Generation
Requirement	The system shall generate PDF reports containing: 1. Timestamped cheating incidents with confidence scores 2. Video metadata (subject, exam type, institution) 3. Model type used (ML/DL)
Source	Project Proposal
Rationale	Provide actionable, auditable evidence of academic integrity violations
Business Rule	Max 3 pages per report
Dependencies	1. MongoDB (video metadata) 2. Institutional branding assets
Priority	High

Table 11: Functional Requirement FR-7 — Multi-Level Admin Dashboard

Identifier	FR-7
Title	Multi-Level Admin Dashboard
Requirement	Provide role-specific dashboards: 1. System Admin: Institution/subscription analytics 2. Institutional Admin: Faculty management + storage usage 3. Faculty: Subject-specific video reports
Source	Project Proposal
Rationale	Tailored interfaces for efficient management
Business Rule	1. JWT role claims enforce access control 2. Data isolation between institution
Dependencies	Next.js (frontend), Django REST (role middleware)
Priority	High

Table 12: Nonfunctional Requirement NFR-1 — Performance

Identifier	NFR-1
Title	Performance
Requirement	The system shall respond to user interactions (e.g., button clicks, page navigation) within ≤ 2 seconds for 95% of requests.
Priority	High

Table 13: Nonfunctional Requirement NFR-2 — Security

Identifier	NFR-2
Title	Security
Requirement	All videos and user data shall be encrypted in transit (HTTPS)
Priority	High

Chapter 3

Use Case Analysis

Chapter 3: Use Case Analysis

This chapter outlines detailed descriptions of key use cases for the system. Each use case represents a specific action or interaction that a user or system component performs. The use cases include processes like user sign-up, login, video processing for both non-premium and premium users, and administrative management tasks such as subscription and account deletion. These use cases are designed to ensure clarity in system functionality, with descriptions of normal flows, alternative scenarios, exceptions, business rules, and assumptions. The goal is to provide a comprehensive understanding of the system's operations from both a user and administrative perspective.

3.1. Use Case Diagram

This section provides a structured representation of the system's functionalities from the perspective of different actors interacting with it. It visually maps out the various use cases, their relationships, and how users (both regular and admin) will interact with the system. This model helps to capture and define the system's functional requirements in a clear and concise manner, outlining each user's role and the actions they can perform within the system. The Use Case Model serves as a blueprint for understanding system behavior and guiding both development and testing phases.

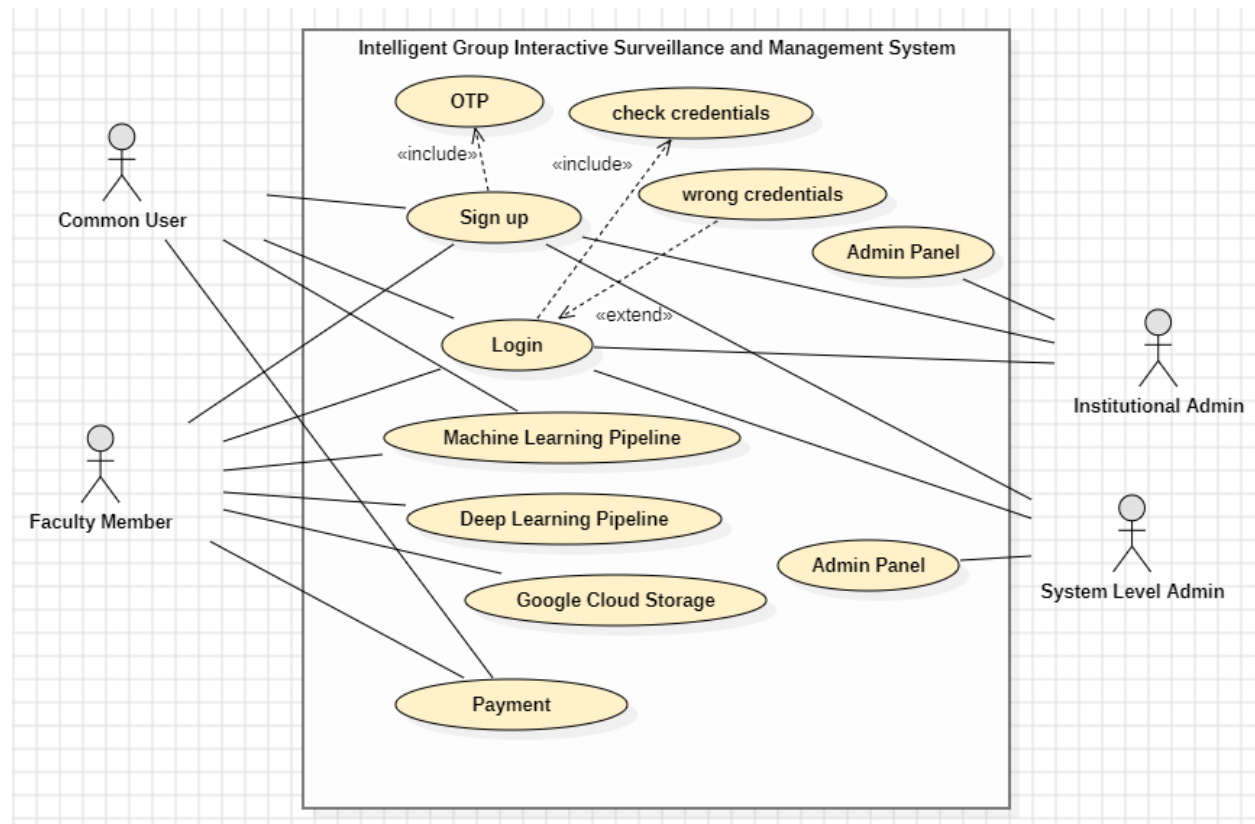


Figure 12: Use Case Diagram of Intelligent Group Interactive Surveillance and Management System

3.2. Use Cases Description

This section provides a detailed explanation of each use case identified in the Use Case Model. It includes critical information such as the actors involved, preconditions, normal and alternative flows, exceptions, business rules, and assumptions for each use case. These descriptions are designed to give a clear understanding of how the system will behave under different scenarios. The goal is to ensure that all possible interactions and system responses are well-defined, allowing for accurate implementation and testing of each functionality.

• Use Case 1: User Sign Up

Table 14: Use Case UC-1 — User Sign Up

Use case ID	UC-1
Use case Name	Sign Up
Actors	Guest User
Description	Allows new users to register for an account via email and OTP verification
Precondition	User does not have an existing account
Post condition	User account is created in MongoDB, OTP is sent to email
Normal Flow	1. User enters email, name, and password → 2. System validates inputs → 3. Sends OTP → 4. User submits OTP → 5. Account is activated.
Alternative Flows	OTP expires → System resends OTP
Exception	a. Email exists → Show "Account already exists" error. b. Weak password → Enforce complexity rules
Business Rules	Password: ≥ 8 chars, 1 special char, 1 number. - OTP valid for 5 minutes.
Assumptions	User has access to the provided email

• Use Case 2: User Log In

Table 15: Use Case UC-2 — User Log In

Use case ID	UC-2
Use case Name	Log In
Actors	Registered User (Non-Premium/Premium), Admin
Description	Authenticates users via email/password and issues JWT tokens
Precondition	User has an active account.
Post condition	User receives JWT tokens (access + refresh); session starts
Normal Flow	1. User enters email/password → 2. System verifies credentials → 3. Generates JWT tokens → 4. Grants access.
Alternative Flows	Refresh token used → Issues new access token
Exception	a. Wrong credentials → Show "Invalid email/password" error. b. Inactive account → Prompt to verify email
Business Rules	Access token expires in 2 hours.
Assumptions	JWT Secret Key is securely Stored

- **Use Case 3: OTP Verification**

Table 16: Use Case UC-3 — OTP Verification

Use case ID	UC-3
Use case Name	OTP Verification
Actors	Newly Registered User
Description	Verifies user's email via OTP
Precondition	OTP sent to user's email
Post condition	User account activated (blocked=False)
Normal Flow	1. User submits OTP → 2. System validates → 3. Unblocks account
Alternative Flows	OTP resend → Restarts 5-minute timer
Exception	a. Max attempts → Lock OTP field for 1 hour
Business Rules	OTP is 6-digit numeric
Assumptions	User has access to the provided email

- **Use Case 4: Video Processing (Common User)**

Table 17: Use Case UC-4 — Video Processing (Common)

Use case ID	UC-4
Use case Name	Video Processing
Actors	Common User
Description	Processes video without storage
Precondition	User has subscription=False
Post condition	Returns classification results only
Normal Flow	1. Upload video → 2. Classify via ML → 3. Return PDF report
Alternative Flows	None
Exception	Unsupported file type → Reject upload
Business Rules	Results expire after 24 hours
Assumptions	User just want to use the system for once

- **Use Case 5: Video Processing (Faculty-Premium)**

Table 18: Use Case UC-5 — Video Processing (Faculty-Premium)

Use case ID	UC-5
Use case Name	Video Upload (Faculty-Premium)
Actors	Faculty User
Description	Uploads exam video for cheating detection with metadata
Precondition	User has subscription=True, faculty_member=True
Post condition	Video stored in GCS, metadata in MongoDB
Normal Flow	1. Upload video → 2. Classify via DL/ML → 3. Store results
Alternative Flows	ML model fallback if DL fails
Exception	a. Corrupted video → Return HTTP 500
Business Rules	Premium users can use both DL/ML models
Assumptions	User wants model with high accuracy

- **Use Case 6: System Admin Subscription Management**

Table 19: Use Case UC-6 — Admin Subscription Management

Use case ID	UC-6
Use case Name	Manage User Subscriptions
Actors	System Admin
Description	Activates/deactivates institutional premium status
Precondition	Admin has is_admin=True
Post condition	Institution's subscription field updated
Normal Flow	1. Select institution → 2. Toggle status → 3. Confirm
Alternative Flows	Bulk update via CSV upload
Exception	None
Business Rules	Triggers email notification to institutional admin
Assumptions	Admins verify user identity before toggling

• Use Case 7: Add Faculty Member

Table 20: Use Case UC-7 — Add Faculty Member

Use case ID	UC-7
Use case Name	Institutional Admin can add Faculty Member
Actors	Institutional Admin
Description	Creates faculty accounts with assigned subjects
Precondition	Admin has subscribed <code>user_seats_left > 0</code>
Post condition	New faculty account created
Normal Flow	1. Enter faculty details → 2. Assign subjects → 3. Send OTP
Alternative Flows	None
Exception	a. Seat limit reached → Deny creation
Business Rules	Max 10 faculty/institution
Assumptions	Faculty email domain matches institution Admin has verified subject/timing data before assignment

• Use Case 8: View Storage Analytics

Table 21: Use Case UC-8 — View Storage Analytics

Use case ID	UC-8
Use case Name	View Storage Analytics
Actors	System Admin
Description	Shows Analytics
Precondition	Premium institution
Post condition	New faculty account created
Normal Flow	1. Request analytics → 2. Display dashboard
Alternative Flows	None
Exception	None
Business Rules	Data refreshes every 6 hours
Assumptions	Admin has institutional authority to view usage data Analytics exclude temporary processing files

• Use Case 9: View Faculty Dashboard

Table 22: Use Case UC-9 — View Faculty Dashboard

Use case ID	UC-9
Use case Name	View Faculty Dashboard
Actors	Faculty Member
Description	Displays uploaded videos, classification results, and subject-wise analytics
Precondition	Faculty is logged in and has uploaded at least 1 video
Post condition	None
Normal Flow	1. Faculty accesses dashboard → 2. System loads videos/results → 3. Displays data
Alternative Flows	Filter by date/subject
Exception	No videos uploaded → Show "Upload your first video" prompt
Business Rules	Data retention: 30 days for non-premium, 1 year for premium
Assumptions	Faculty has basic technical literacy to navigate filters

• Use Case 10: Assign Subject to Faculty

Table 23: Use Case UC-10 — Assign Subject to Faculty

Use case ID	UC- 10
Use case Name	Assign Subject to Faculty
Actors	Institutional Admin
Description	Institutional admin assign subject to the faculty member
Precondition	Faculty account exists, admin has subscription=True
Post condition	Faculty's subjects array updated in MongoDB
Normal Flow	1. Admin selects faculty → 2. Adds subject/timing → 3. Confirms
Alternative Flows	Bulk assign via CSV upload
Exception	a. Timing conflict → Reject with "Slot occupied"
Business Rules	Max 5 subjects/faculty
Assumptions	1. Faculty availability data is up-to-date 2. Subjects/timings follow institutional standards 3. Admin has authority to assign subjects

• Use Case 11: Generate PDF Report

Table 24: Use Case UC-11 — Generate PDF Report

Use case ID	UC-11
Use case Name	Generate PDF Report
Actors	Faculty/Common User
Description	Creates cheating incident report
Precondition	Video processed and classified
Post condition	Downloadable PDF generated
Normal Flow	1. Request report → 2. Compile timestamps → 3. Download
Alternative Flows	None
Exception	No cheating detected → "Clean" report
Business Rules	PDF retention Max 3 pages per report
Assumptions	1. Classification results are accurate and verifiable 2. Users have PDF reader software

• Use Case-12: View Stored Video (Faculty User)

Table 25: Use Case UC-12 — View Stored Video

Use case ID	UC-12
Use case Name	View Stored Video
Actors	Faculty User
Description	Allows faculty users to view their stored videos in full-screen mode
Precondition	User is authenticated and subscribed
Post condition	Video displays in full-screen mode with playback controls
Normal Flow	1. User clicks thumbnail in video history list 2. System: a. Fetches video URL b. Opens video player component in full-screen mode c. Streams video from GCS 3. User sees: a. Video in full-screen b. Classification result
Alternative Flows	User presses esc exits full-screen but continues playback
Exception	Buffering >5s → Show loading spinner
Business Rules	Full-screen requires user gesture (no auto-fullscreen)
Assumptions	Modern browsers support Fullscreen API

• Use Case 13: Delete Stored Video (Faculty User)

Table 26: Use Case UC-13 — Delete Stored Video (Faculty User)

Use case ID	UC-13
Use case Name	Delete Stored Video
Actors	Subscribed Faculty User
Description	Allows premium users to permanently delete their stored videos and metadata
Precondition	1. User is authenticated 2. User has active subscription 3. Target video exists in user's history
Post condition	Video file removed from GCS and metadata deleted from MongoDB
Normal Flow	1. User requests video list 2. System displays list with delete buttons 3. User selects video to delete 4. System confirms deletion 5. Backend removes: a. GCS file b. MongoDB record
Alternative Flows	User cancels deletion → Returns to video list
Exception	Video not found
Business Rules	Only video owner can delete (verify user_id in MongoDB)
Assumptions	User may want to free up storage space

• Use Case-14: System Admin Removes Institutional Admin

Table 27: Use Case UC-14 — System Admin Removes Institution Admin

Use case ID	UC-14
Use case Name	System Admin Removes Institution Admin
Actors	System Admin
Description	Deletes an institutional admin
Precondition	1. Admin is authenticated 2. Institutional Admin exists in DB
Post condition	User account and all related videos deleted
Normal Flow	1. Admin selects institution → 2. Confirms deletion → 3. System cleans up
Alternative Flows	Admin exports user data before deletion → Delay deletion by 24h
Exception	a. Active subscriptions → Block deletion until resolved
Business Rules	Admin cannot delete other admin
Assumptions	Performed rarely for compliance reasons

- **Use Case-15: Logout (All Users)**

Table 28: Use Case UC-15 — Logout (All Users)

Use case ID	UC-15
Use case Name	Logout
Actors	Any User
Description	Terminates authenticated session
Precondition	User is logged in
Post condition	JWT invalidated and session closed
Normal Flow	1. User clicks logout 2. Frontend: a. Discards JWT b. Redirects to login
Alternative Flows	. "Remember me" enabled → Keep session cookie
Exception	Network failure → Retry silently
Business Rules	Tokens expire in 24h
Assumptions	Mobile users may stay logged in longer

Chapter 4

System Design

Chapter 4: System Design

This chapter outlines the system design for the proposed web application, detailing the architectural and structural elements that ensure efficient functionality and maintainability. It provides a comprehensive view of how various components and processes interact within the system. Starting with the overall architecture, the section includes several key diagrams that illustrate the design, including the Domain Model, Entity Relationship Diagram, and Class Diagram. These diagrams offer insight into the relationships between various entities, the data flow, and the components within the system. Additionally, diagrams such as the Sequence/Collaboration Diagram, Activity Diagram, and State Transition Diagram describe the dynamic aspects of the system, highlighting the sequence of operations and transitions within the user and system interactions. The Component Diagram and Deployment Diagram further explain the distribution of components and their deployment across different environments. Together, these diagrams provide a clear, detailed roadmap for the system's construction and operation.

4.1. Architecture Diagram

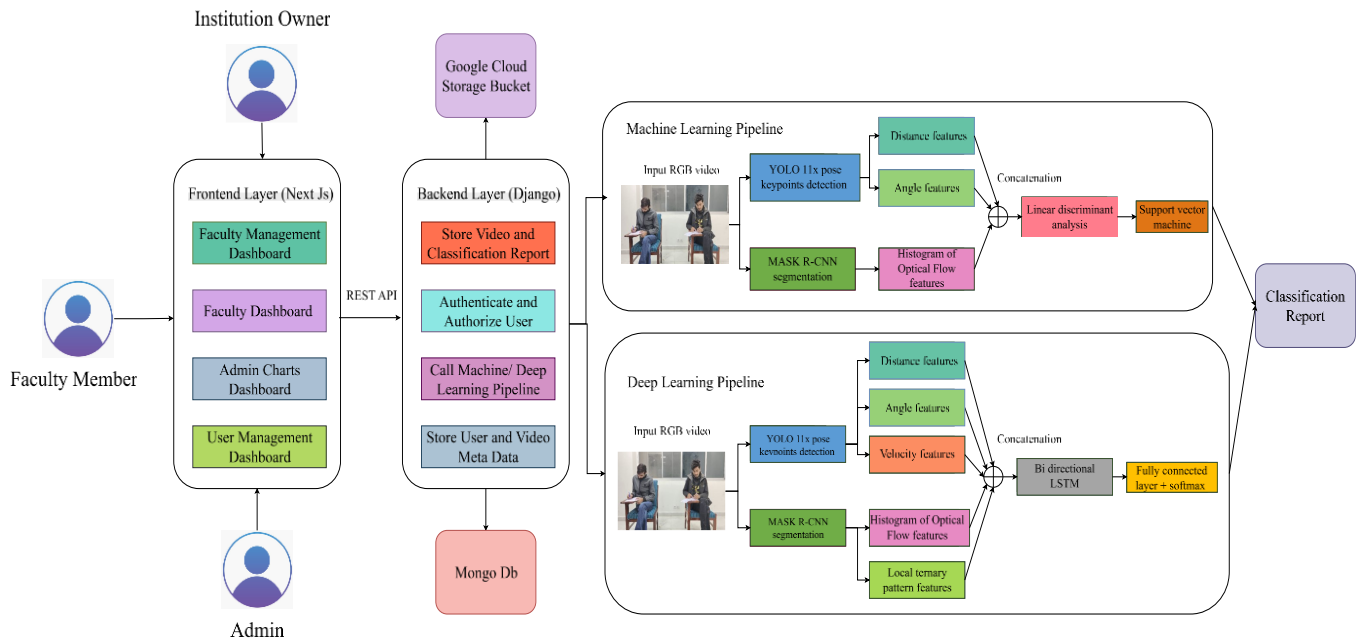


Figure 13:Architecture Diagram of Intelligent Group Interactive Surveillance and Management System

4.2. Domain Model

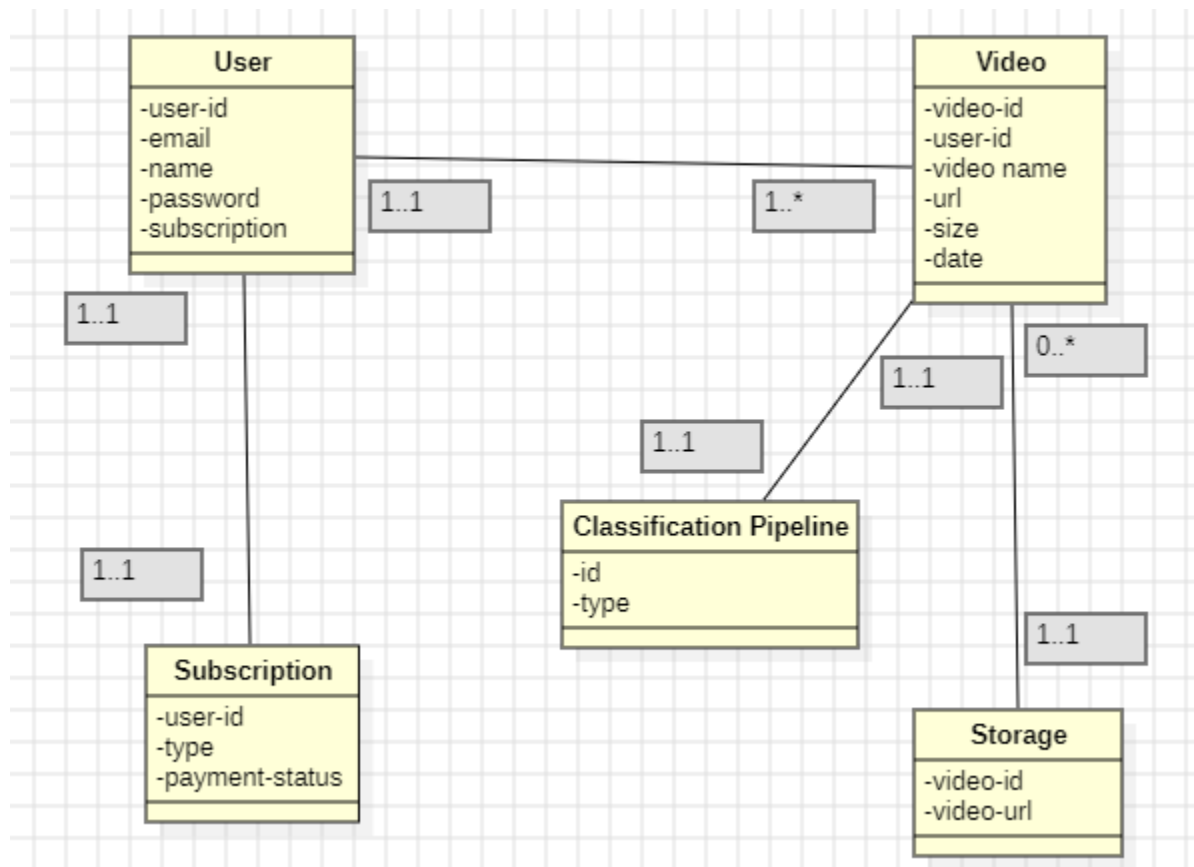


Figure 14:Domain Model Diagram of Intelligent Group Interactive Surveillance and Management System

4.3. Entity Relationship Diagram with data dictionary

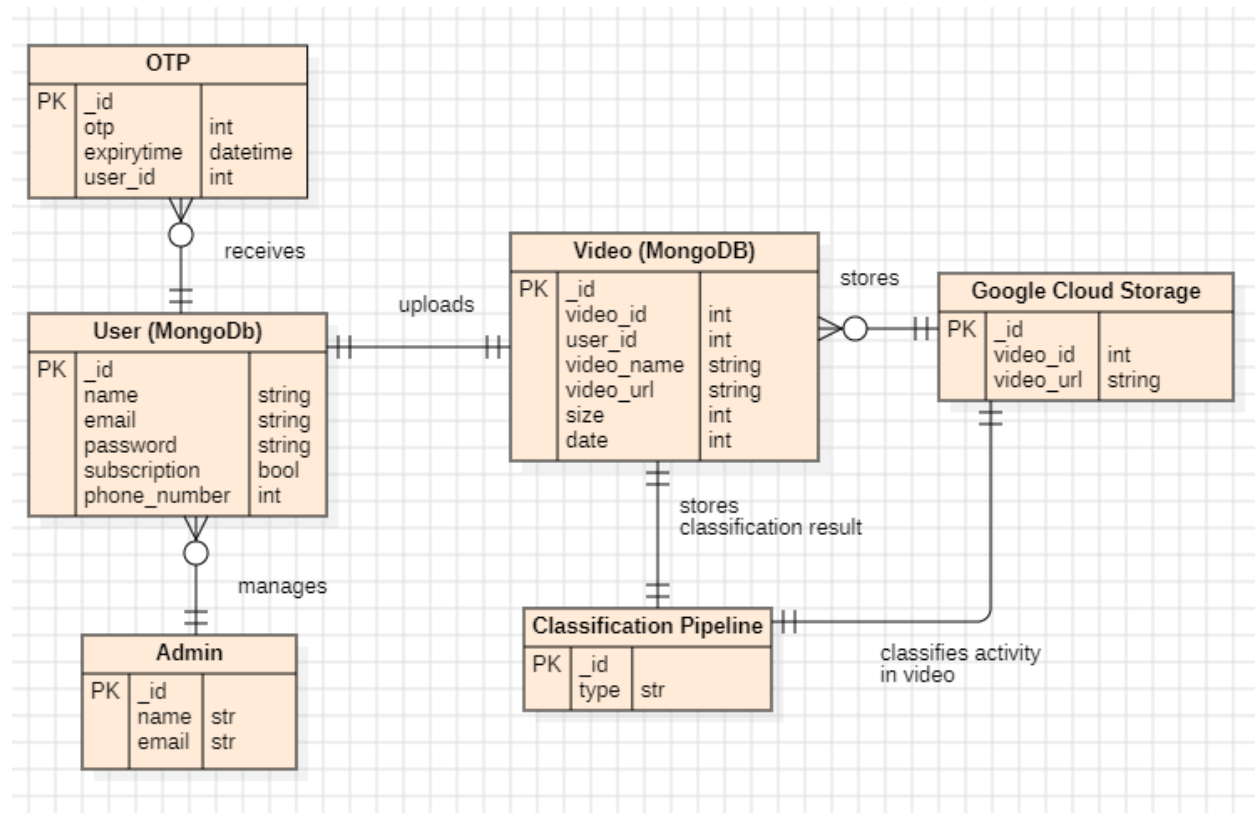


Figure 15:ER-Diagram of of Intelligent Group Interactive Surveillance and Management System

4.4. Class Diagram

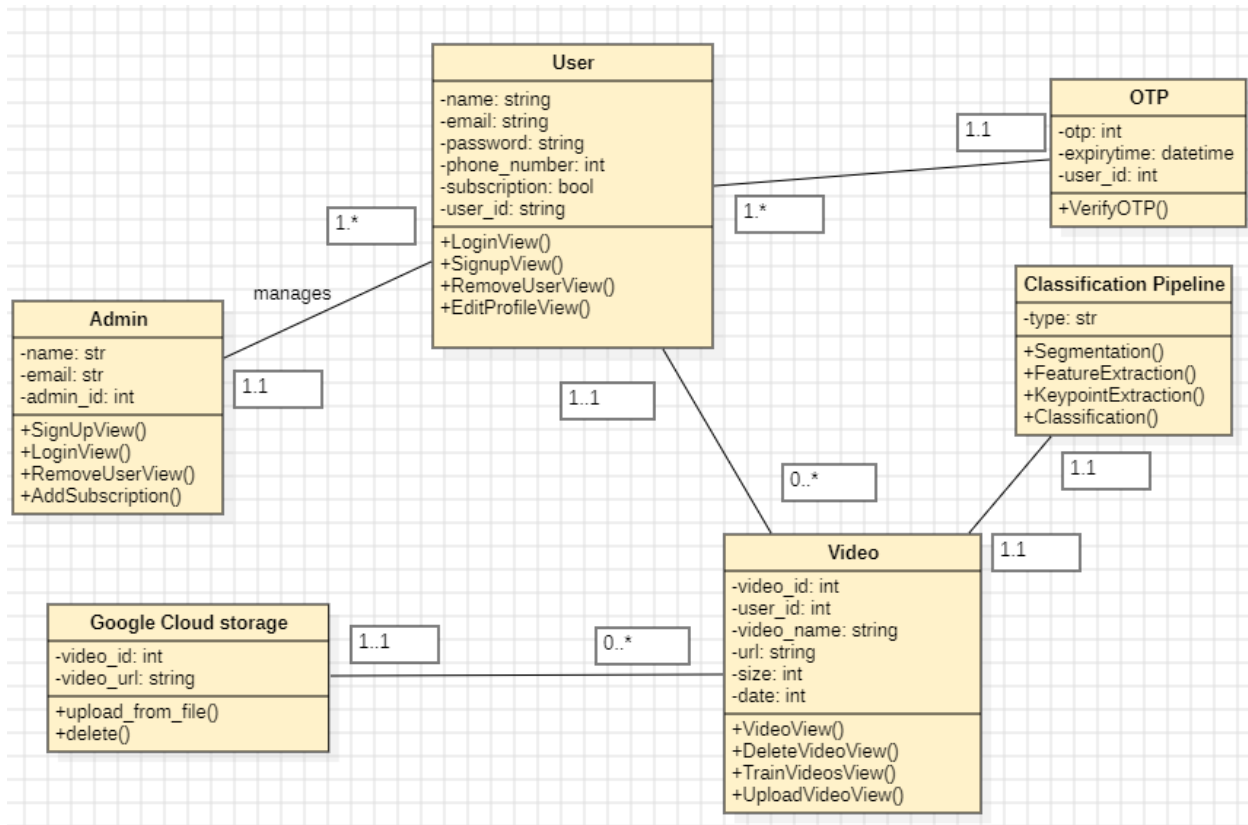


Figure 16:Class Diagram of Intelligent Group Interactive Surveillance and Management System

4.5. Sequence / Collaboration Diagram

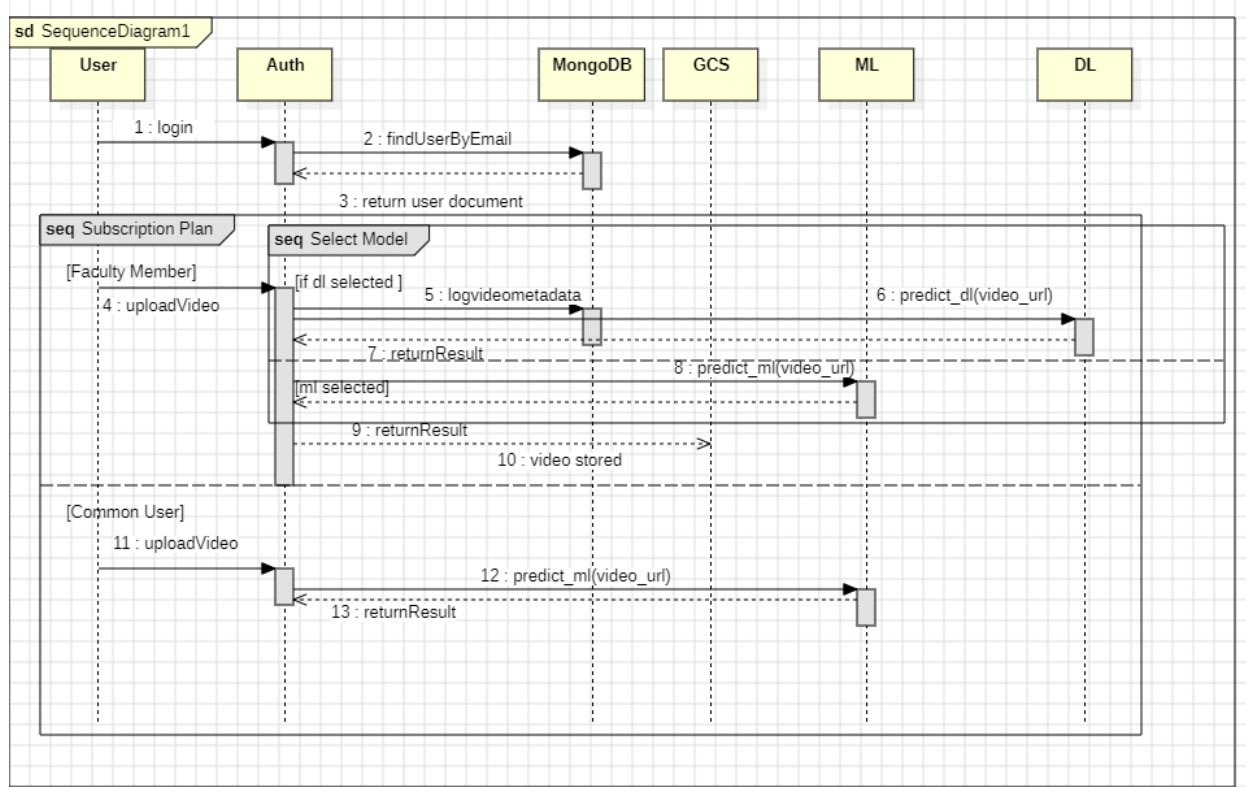


Figure 17: Sequence Diagram for common user and faculty member of Intelligent Group Interactive Surveillance and Management System

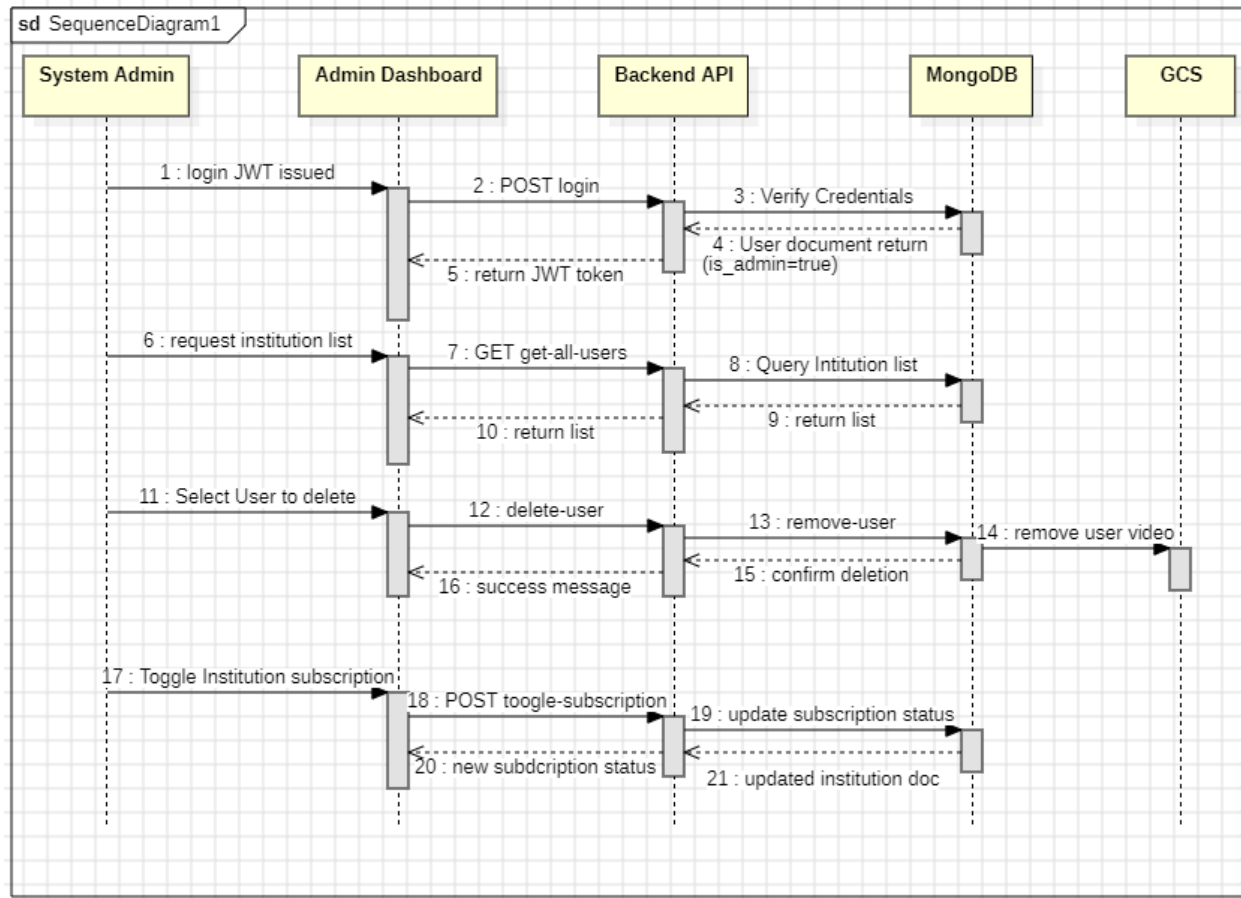


Figure 18: Sequence Diagram for System Admin of Intelligent Group Interactive Surveillance and Management System

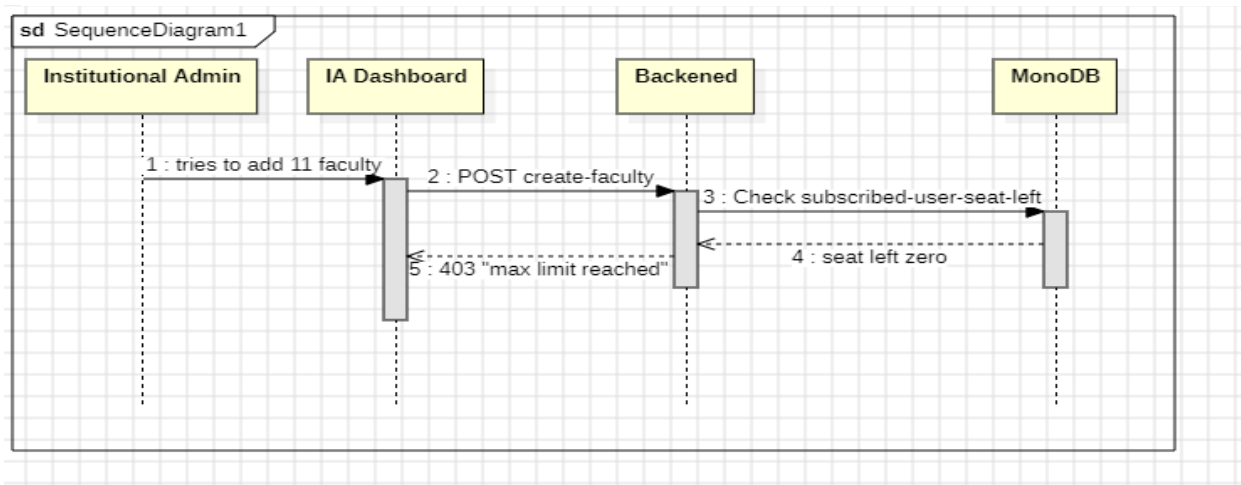


Figure 19: Sequence Diagram for Seat limit reached of Intelligent Group Interactive Surveillance and Management System

4.6. Activity Diagram

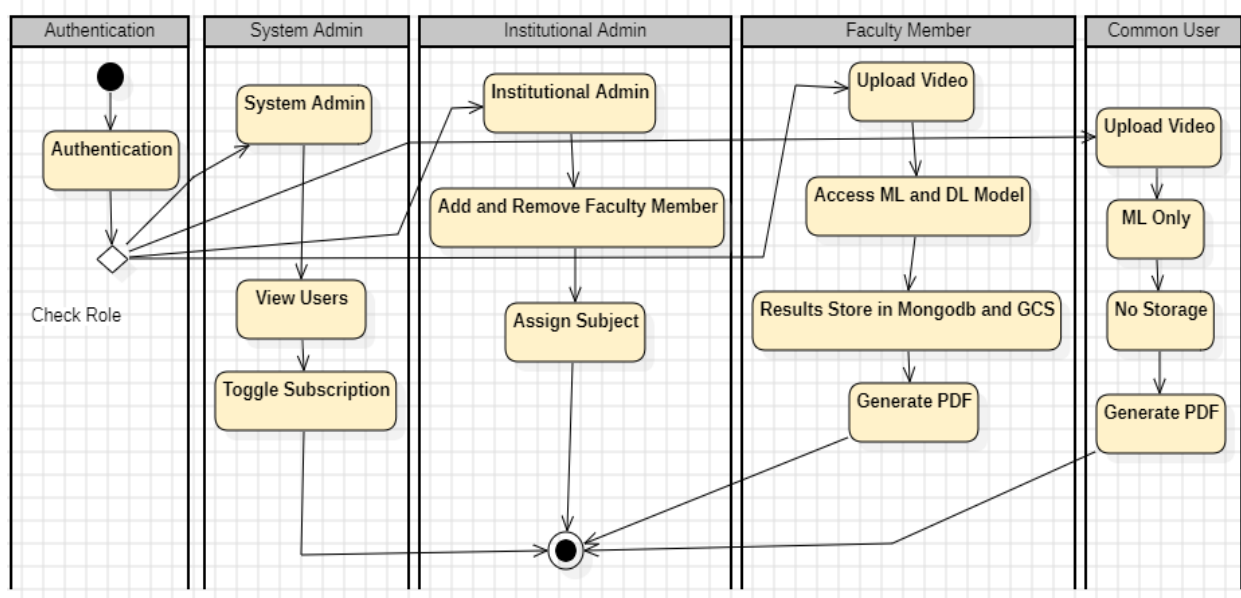


Figure 20: Activity Diagram of Intelligent Group Interactive Surveillance and Management System

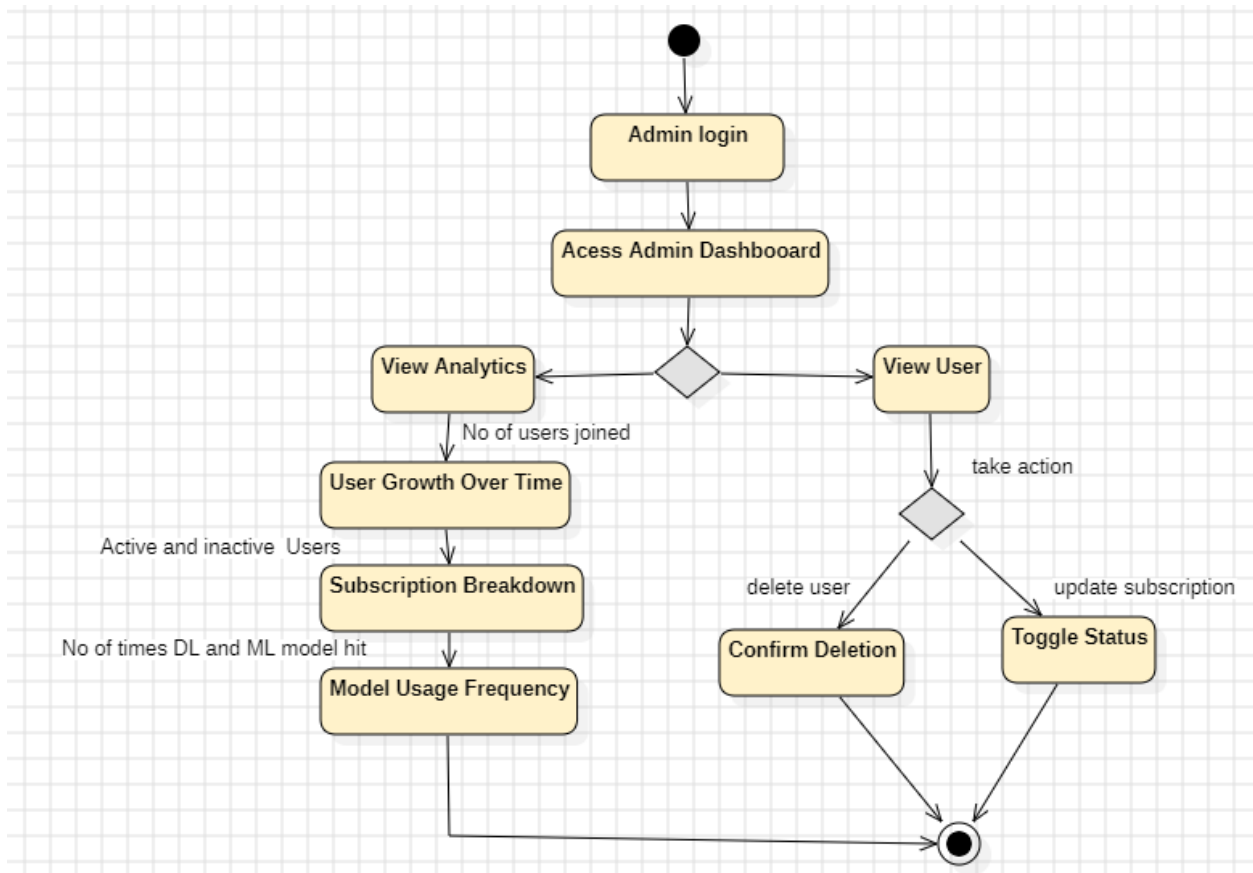


Figure 21: System Admin Activity Diagram of Intelligent Group Interactive Surveillance and Management System

4.7. State Transition Diagram

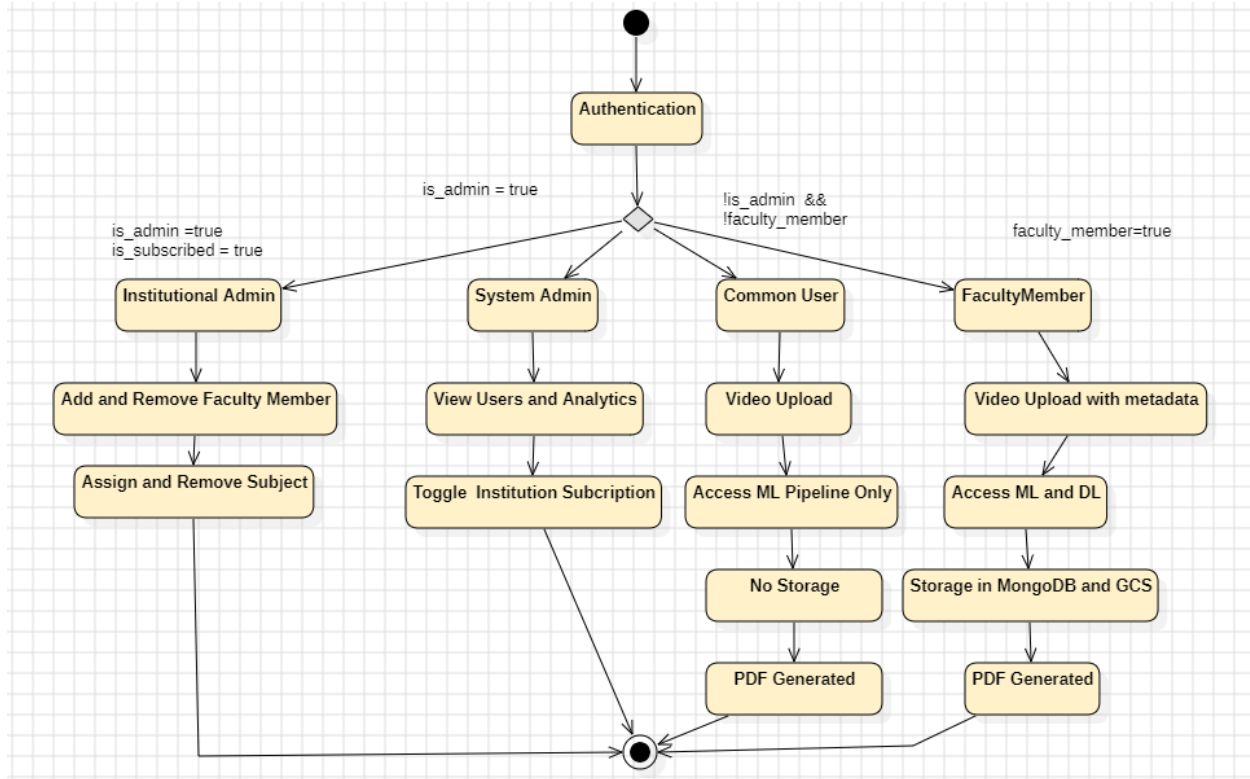


Figure 22:State transition diagram of Intelligent Group Interactive Surveillance and Management System

4.8. Component Diagram

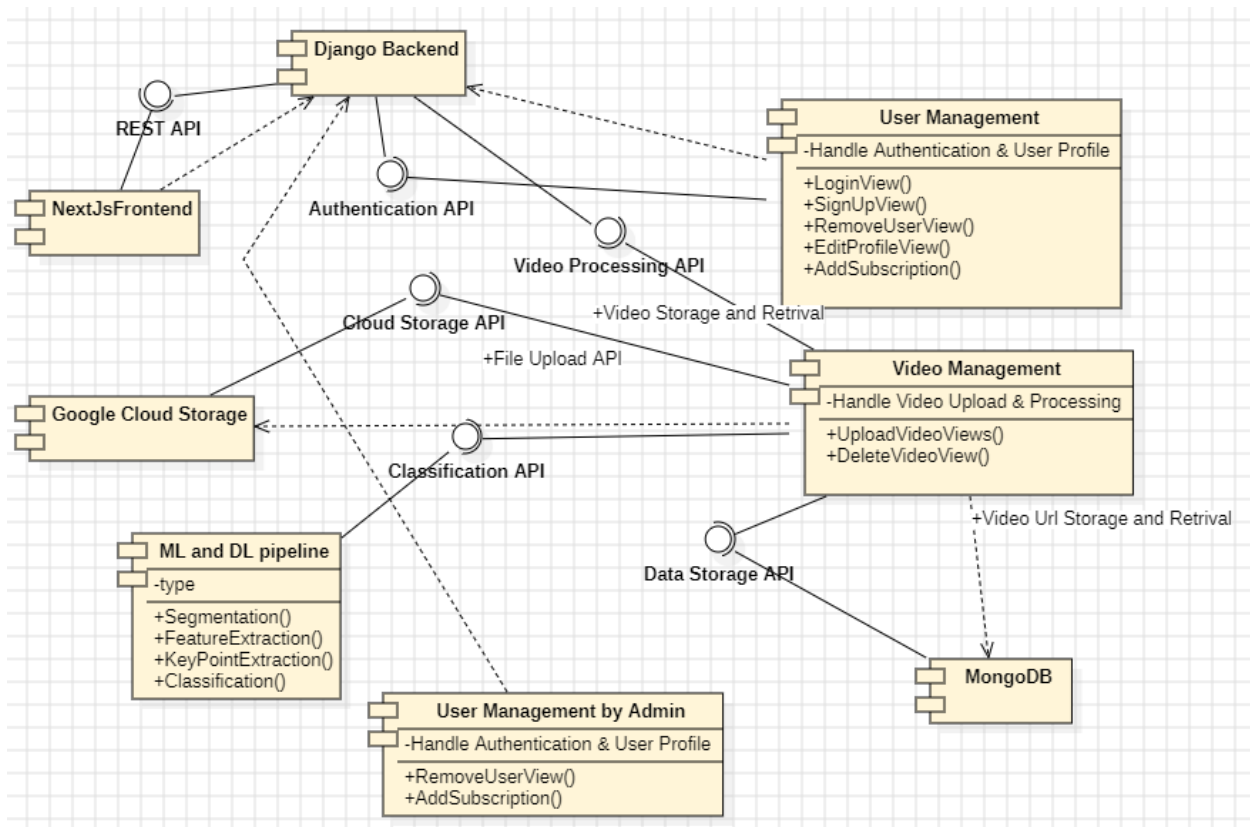


Figure 23:Component Diagram of Intelligent Group Interactive Surveillance and Management System

4.9. Deployment Diagram

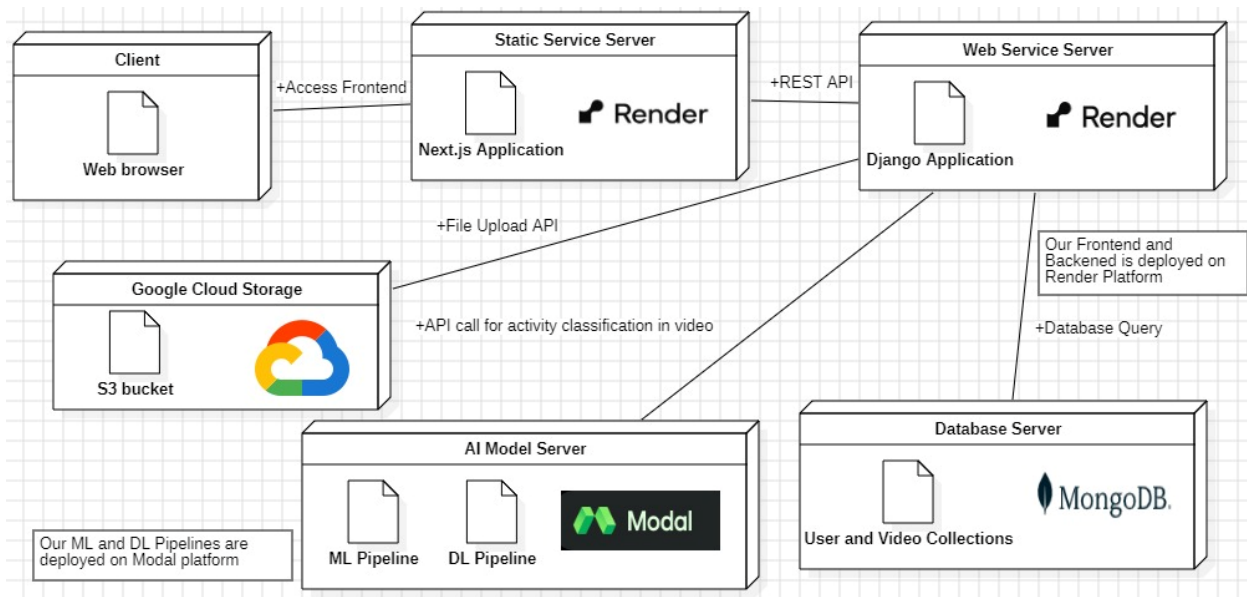


Figure 24:Deployment Diagram of Intelligent Group Interactive Surveillance and Management System

Chapter 5

Implementation

Chapter 5: Implementation

This chapter focuses on the core aspect of our project, specifically the AI components. It provides an in-depth explanation of our machine learning and deep learning classification pipelines, covering segmentation, keypoints detection, feature extraction, and classification processes. Additionally, it discusses the components, libraries, web services, and stubs related to the entire project, including the web application. The chapter also outlines the tools and techniques used, the deployment environment, and highlights best practices, coding standards, and version control strategies.

5.1. Datasets description

Evaluation of our two pipelines was performed using two datasets: the Student Cheating Dataset [1] and our custom-made dataset. Since machine learning models require a large amount of data for proper training and optimal performance, we combined both datasets and performed 3-fold cross-validation. After identifying the best parameters for our machine learning and deep learning models, we saved the weights of the models that achieved the highest accuracy. For deployment, we selected the model weights from the fold with the highest validation accuracy for inference. The details of these datasets are as follows:

5.1.1. Students cheating dataset

The Student Cheating Dataset contains 37 RGB video sequences, captured using a Canon 70D sensor camera, featuring eight unique actors performing cheating activities in pairs within a classroom setting. It consists of five classes: *passing papers*, *looking at other's work*, *using cheat sheets*, *using cellular devices*, and *doing own work*. The sensor recorded 24 frames per second, and the image size was 1920 1080 pixels. For our experiments, we only used video sequences corresponding to the classes: *passing paper*, *looking at other's work* and *doing own work* from the dataset. Figure 12 shows some samples from the student cheating dataset.



Figure 25:Some samples from the Students Cheating dataset

5.1.2. Custom dataset

We also created a custom student cheating dataset to gather additional data for model training. This dataset was generated by recording RGB videos of 2 students performing one of three possible activities in a classroom environment: *passing papers*, *looking at another student's work*, and *doing their own work*. The videos were 5 to 8 seconds in length, with a resolution of 1280×720 pixels. In total, we recorded approximately 74 video sequences. Figure 13 shows sample frames from our custom dataset.



Figure 26: Some samples from our custom dataset

5.2. Segmentation

In order for accurate segmentation of and removal of irrelevant individuals, we use Mask R-CNN [8], which performs instance-level segmentation by generating separate masks for each person. It extends Faster R-CNN [9] by adding a parallel branch for mask prediction. The overall loss function L for training Mask R-CNN is defined as a multi-task loss combining classification loss L_{cls} , bounding box regression L_{box} , and mask prediction loss L_{mask} . The classification loss L_{cls} is computed as:

$$L_{cls} = -\sum_c y_c \log(p_c) \quad (1)$$

Where y_c is the ground-truth label for class c (1 if the RoI belongs to class c , 0 otherwise), and p_c is the predicted probability for class c . The mask prediction loss L_{mask} is defined as the average binary cross-entropy loss over the predicted mask computed for each RoI and ground-truth class k :

$$L_{mask} = -\frac{1}{m^2} \sum_{1 \leq i,j \leq m} [y_{ij} \log(\hat{y}_{ij}^k) + (1 - y_{ij}) \log(1 - \hat{y}_{ij}^k)] \quad (2)$$

Where m is the resolution of the predicted mask, y_{ij} is the ground-truth binary mask value at pixel (i, j) , and $\hat{y}_{ij}^k$ is the predicted mask value for class k at pixel (i, j) .

For our experiments, we used a pre-trained Mask R-CNN model trained on the COCO dataset. To balance segmentation accuracy and efficiency, input RGB frames were resized to an optimal resolution and a confidence threshold was applied to extract gray scale silhouettes. Figure 14. shows segmentation results for our experiments.



Figure 27: Segmentation Results from our experiments. (a) shows original frames and (b) shows segmented gray scale silhouettes.

5.3. Keypoints detection

For keypoints detection across frames of interacting persons, we use YOLOv11-Pose [10] from the YOLO framework. YOLOv11-Pose builds on YOLOv8's loss structure. Detection loss includes bounding box loss L_{box} (IoU or Smooth L1), classification loss L_{cls} (Binary Cross-Entropy), and Distribution Focal Loss L_{dfl} (DFL). For keypoints detection, additional losses are used for keypoint localization loss $L_{kpt_{loc}}$ (Smooth L1) and keypoint visibility loss $L_{kpt_{vis}}$ (Binary Cross-Entropy), combined in a weighted total loss computed as:

$$L_{total} = \lambda_{box} * L_{box} + \lambda_{cls} * L_{cls} + \lambda_{dfl} * L_{dfl} + \lambda_{kpt_{loc}} * L_{kpt_{loc}} + \lambda_{kpt_{vis}} * L_{kpt_{vis}} \quad (3)$$

Where λ_{box} , λ_{cls} , λ_{dfl} , $\lambda_{kpt_{loc}}$ and $\lambda_{kpt_{vis}}$ are task-specific weighting factors. For our experiments, we used YOLOv11n-Pose, the smallest YOLOv11-Pose variant, due to its fast-

processing speed with ByteTrack tracker for correct keypoints assignment across frames. ByteTrack associates persons with unique IDs across frames in two stages. First, high-confidence detections are matched to existing tracks using Intersection over Union (IoU). Second, low-confidence detections are re-associated to recover potentially missed objects, reducing identity switches. Out of the 17 keypoints detected by YOLOv11-Pose, we used 7 keypoints namely Nose, left and right shoulders, wrists and elbows. Figure 15 visualizes keypoints detection and tracking for our experiments.

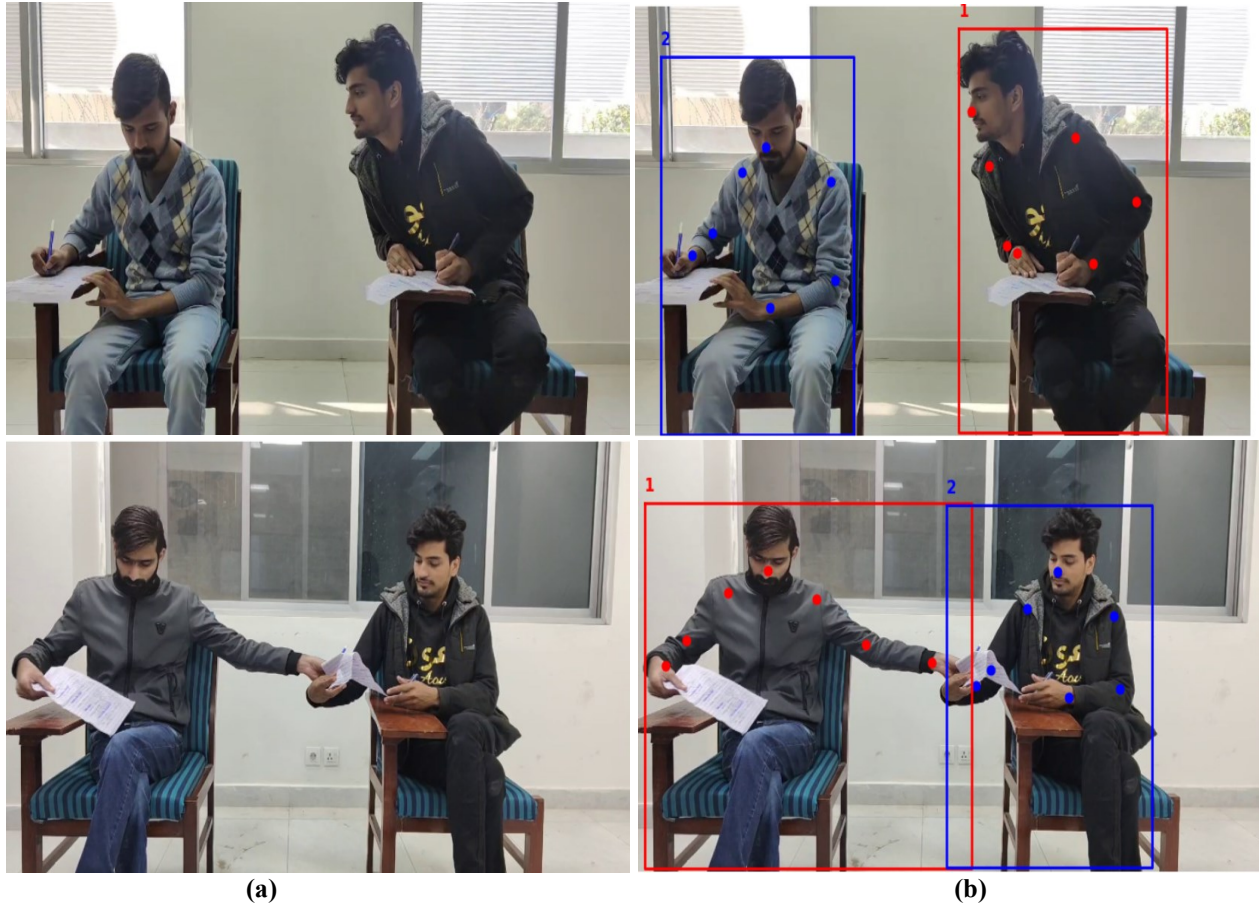


Figure 28: Keypoints detection and human tracking by applying YOLOv11-Pose (a) shows original and (b) shows keypoint detected and human tracked frames.

5.4. Feature Extraction

Our system captures both spatial and temporal features from gray scale silhouettes and keypoints. This ensures the extraction of features essential for accurate classification. The details are provided below:

5.4.1. Local ternary pattern features

For the spatial features of grayscale silhouettes, we extracted Local Ternary Pattern (LTP) features, which capture local texture information by encoding the relationships between pixel intensities in a neighborhood. The LTP algorithm extends the Local Binary Pattern (LBP) by introducing a ternary encoding scheme, which divides pixel differences into three categories: positive, negative, and neutral. This allows for more robust feature representation, especially in scenarios with varying illumination or noise. Given a grayscale image I and a center pixel I_c , the LTP code for a neighboring pixel I_p is computed as follows:

$$LTP(I_p) = \begin{cases} 1, & \text{if } I_p - I_c > \tau \\ -1, & \text{if } I_p - I_c < -\tau \\ 0, & \text{otherwise} \end{cases} \quad (4)$$

Where τ is a predefined threshold. The positive LTP and negative LTP codes are processed separately to generate histograms. These histograms are concatenated to form the final feature vector. Figure 16 shows a visualization of local ternary pattern features for our dataset.

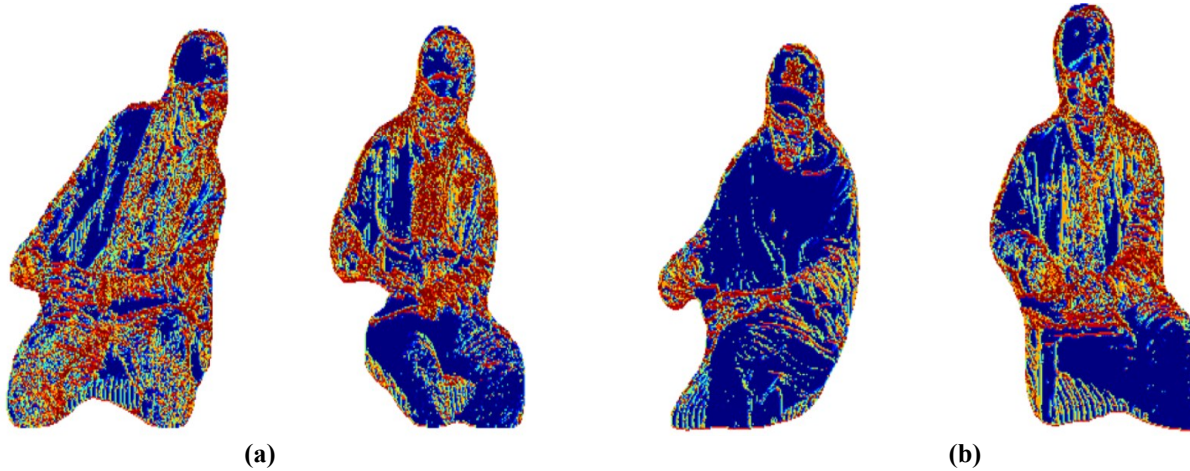


Figure 29: LTP features for (a) looking at other's work and (b) doing own work activities

5.4.2. Histogram of Optical Flow (HOF) features

To capture temporal motion information in video sequences, we extracted Histogram of Optical Flow (HOF) features from gray scale silhouettes. These features encode the motion patterns between consecutive frames by analyzing the displacement of pixels over time. The optical flow is computed using the Farneback [11] method, which estimates the dense motion vectors

between two consecutive frames. The flow vectors F are decomposed into their magnitude $|F|$ and angle θ , defined as:

$$|F| = \text{sqrt}(F_x^2 + F_y^2) \quad (5)$$

$$\theta = \text{arctan2}(F_y, F_x) \quad (6)$$

Where F_x and F_y are the horizontal and vertical components of the flow vectors, respectively. The angles θ are binned into predefined number of bins (covering 0° to 360°), and the corresponding magnitudes are used to weight the histogram. This results in a feature vector for each frame pair, representing the distribution of motion directions weighted by their magnitudes. Figure 17. shows a visualization of HOF features extracted for our experiments.



Figure 30: HOF features for (a) passing paper and (b) doing own work activities. It can be seen that (a) has very high optical flow movement across frames as compared to (b)

5.4.3. Distance Features

To capture spatial relationships between keypoints, we extracted intra-person and inter-person distance features. These features mostly describe the overall body shape and spacing between people, helping the system understand how far or close certain parts are during different interactions such as doing own work versus passing paper. Euclidean distance between two keypoints $P_1(x_1, y_1)$ and $P_2(x_2, y_2)$ is computed as:

$$\text{distance} = \text{sqrt}((x_2 - x_1)^2 + (y_2 - y_1)^2) \quad (7)$$

Instead of computing distances for all possible joint pairs, we focus on specific intra-person and inter-person distances to extract only meaningful spatial information. This selective approach ensures a compact yet informative representation of spatial relationships. Figure 18 shows a visualization of distance features of different interactions from our experiments.

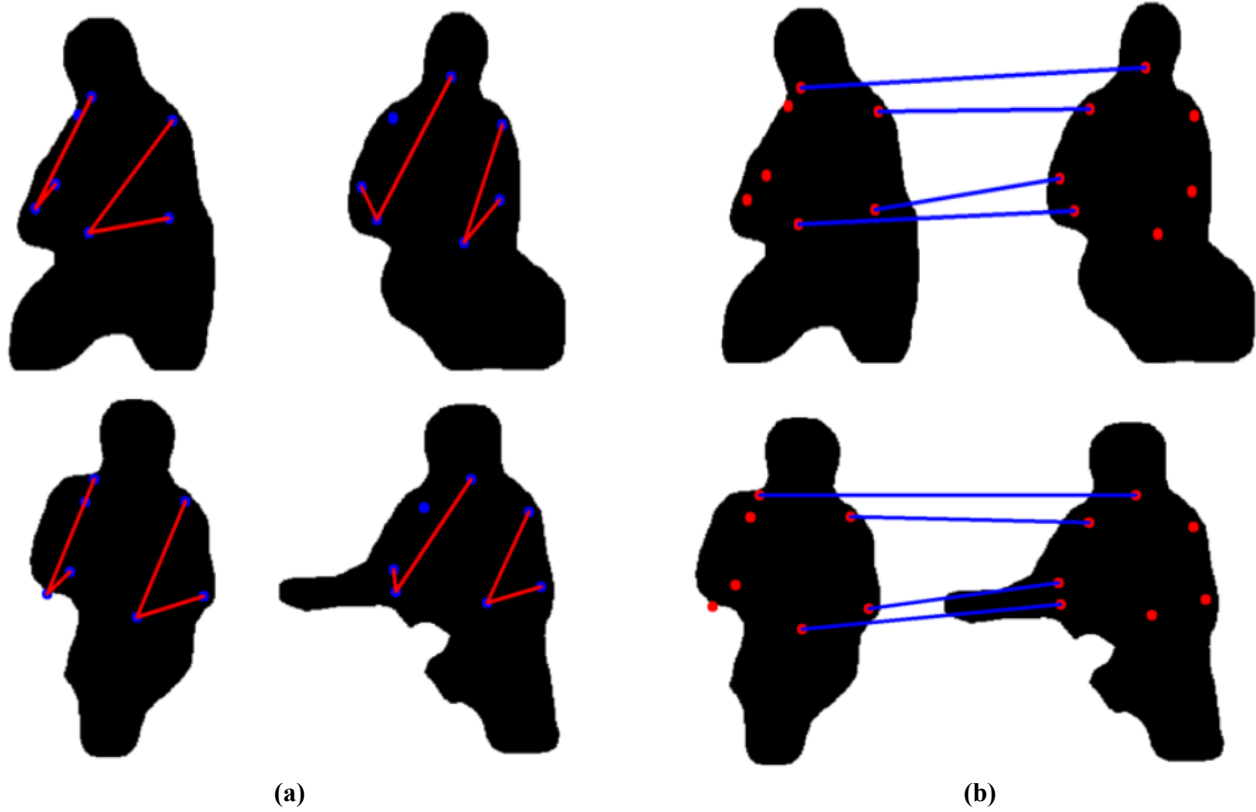


Figure 31: (a) Intra distance and (b) inter distance features for passing paper and doing own work activities.

To capture orientation information between keypoints, we extracted intra-person and inter-person angle features. These features capture how the bodies of the interacting persons are oriented or angled, which helps tell apart actions that may have similar distances but different joint directions, like passing paper versus looking at other's work. The angle θ between two keypoints $P_1(x_1, y_1)$ and $P_2(x_2, y_2)$ is computed as:

$$\theta = \arctan2(y_2 - y_1, x_2 - x_1) \quad (8)$$

Similar to distance features, we focus on specific intra-person and inter-person mappings to keep only the necessary information. This selective extraction ensures that only angle features are extracted for activity recognition. Figure 18 shows a visualization of angle features for our experiments.

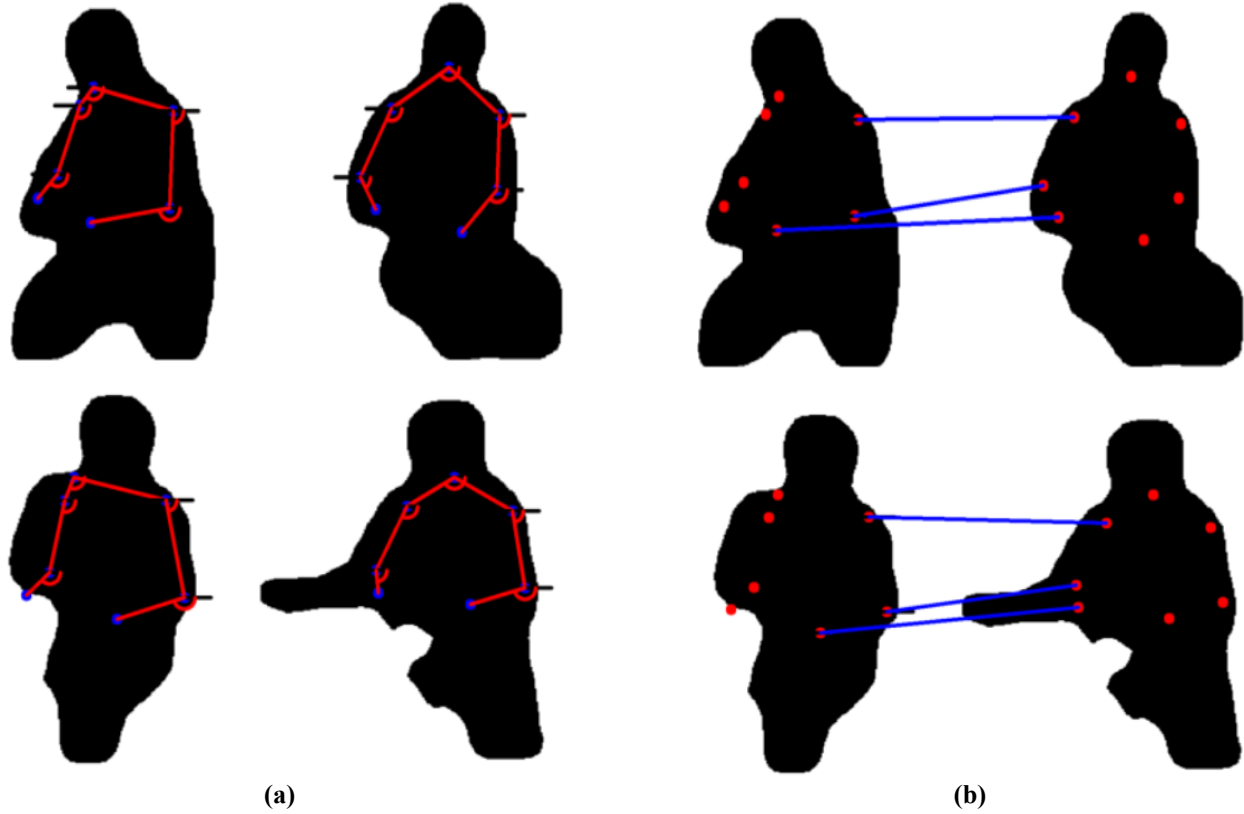


Figure 32: (a) Intra angle and (b) inter angle features for passing paper and doing own work activities.

Table 20 shows the intra and inter mappings we created for measuring angles and distances, with arrows indicating direction. For intra-distances/angles, both sides of the arrow represent the same person's keypoints. For inter-distances/angles, the left side corresponds to the left person's keypoint, while the right side corresponds to the right person's keypoint in the interaction.

Table 29: Mappings between keypoints for measuring angles and distances

Calculation type	Distance/ Angle type	Keypoints mapping	No of distances/ angles
Distance	Intra	$N^1 \rightarrow RW^2$ $LS^3 \rightarrow LW^4$ $RE^5 \rightarrow RW^6$ $LE^7 \rightarrow LW$	8 (4 x 2)
Distance	Inter	$LS \rightarrow RS$, $LW \rightarrow RW$ $LE \rightarrow RE$ $N \rightarrow N$	4
Angles	Intra	$N \rightarrow RS, LS$ $RS \rightarrow RE$ $LS \rightarrow LE$ $RE \rightarrow RW$ $LE \rightarrow LW$	12 (6 x 2)
Angles	Inter	$LW \rightarrow RW$ $LE \rightarrow RE$ $LS \rightarrow LE$	3

¹ Nose, ² Right wrist, ³ Left shoulder, ⁴ Left wrist ⁵ Right elbow, ⁶ Right wrist, ⁷ Left elbow

5.4.4. Velocity features

To capture temporal motion dynamics, we extracted velocity features from each person's keypoints of consecutive frames. The velocity for a keypoint between two consecutive frames is computed as the Euclidean distance between its coordinates in the current frame (x_t, y_t) and the previous frame (x_{t-1}, y_{t-1}) :

$$velocity = \sqrt{(x_t - x_{t-1})^2 + (y_t - y_{t-1})^2} \quad (9)$$

These velocity features encode the motion patterns of keypoints over time which can be helpful for activity classification. By combining spatial and temporal features, we achieve a more comprehensive representation of human motion during each interaction, improving the accuracy of activity classification in our system. Figure 19. shows a visualization of velocity features for our experiments.

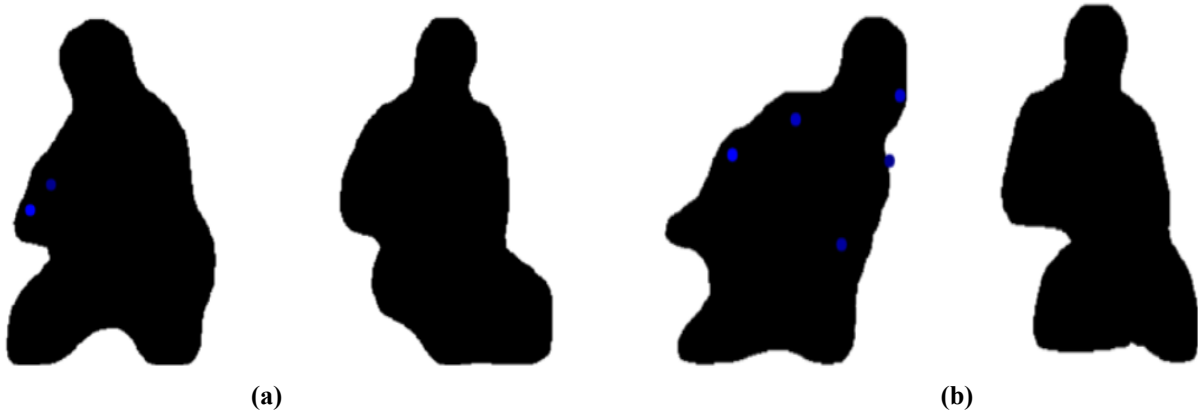


Figure 33: Velocity heatmap for (a) doing own work and (b) looking at other's work activities. Brighter keypoints indicate higher velocity. In (b), the person on the left shows higher velocity. In (a), movement is mainly concentrated around the handwriting areas

5.5. Classification

5.5.1. Machine Learning Model

For our machine learning pipeline, we utilized three features—HOF, Angle, and Distance—selected from the five features described earlier. For feature analysis and optimization, we applied Linear Discriminant Analysis, and for classification, we employed a Support Vector Machine. Further details are provided below. Figure 20 illustrates the architecture of our machine learning pipeline.

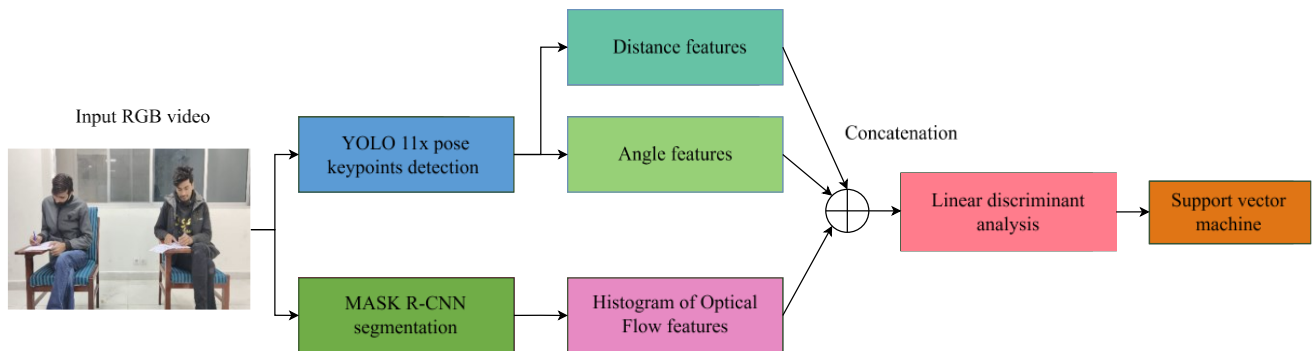


Figure 34: Architecture of Machine Learning pipeline

5.5.1.1. Feature Analysis and Optimization

After the extraction of relevant features, the feature vectors are standardized to ensure comparable scales, preventing any feature from dominating the analysis. Then for feature analysis and optimization, we apply Fisher's Linear Discriminant Analysis (LDA) [12], which

maximizes the ratio of between-class variance to within-class variation $J(W)$ by combining features in a linear fashion, defined as:

$$J(W) = \frac{|W^T S_B W|}{|W^T S_W W|} \quad (10)$$

Where W is the projection matrix (set of eigenvectors), S_B is the between-class scatter matrix, and S_W is the within-class scatter matrix. The projection matrix W is determined by solving the generalized eigenvalue problem defined as:

$$S_B W = \lambda S_W W \quad (11)$$

Where λ represents the eigenvalues for each projection direction defined by the eigenvectors in W . This process reduces the feature space, enhancing class separability and improving classification performance. In our experiments, applying LDA results in a feature vector with a dimensionality of $n - 1$ classes, corresponding to 2 dimensions for 3 classes. Fig. 21 visualizes the separation of samples using LDA for our experiments, showing good class separation.

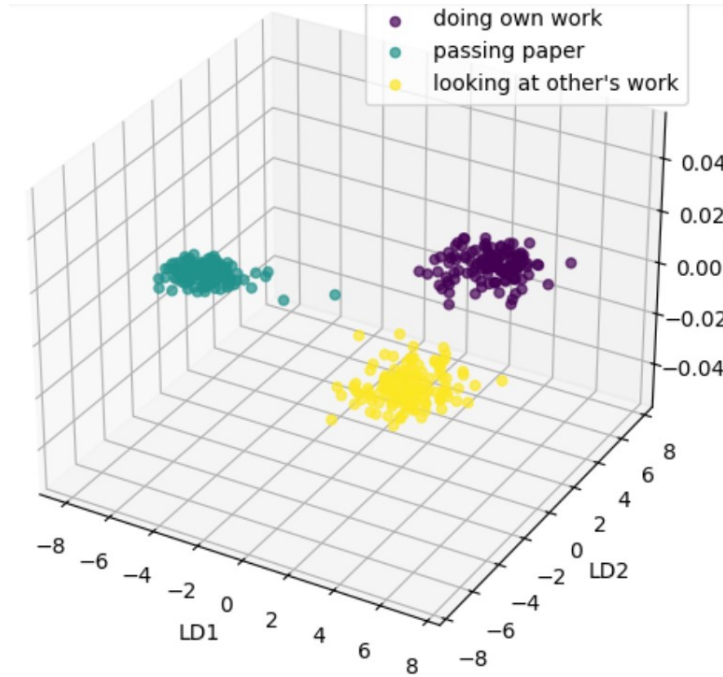


Figure 35: LDA feature optimization

5.5.1.2. Support Vector machine classification

The final stage is classification using a multi-class Support Vector Machine (SVM), which finds the optimal hyperplane to separate classes in the feature space. After LDA optimization, we apply SVM to the optimized feature vectors and use grid search to select the best kernel—linear,

polynomial, RBF, or sigmoid—along with optimal hyperparameters.

For our experiments, the sigmoid kernel $K(x, y)$ was identified as the best kernel function defined as:

$$K(x, y) = \tanh(\gamma \cdot x^T \cdot y + c) \quad (12)$$

Where γ is a hyperparameter that controls the slope of the sigmoid function, $x^T \cdot y$ is the dot product of points x and y . and c is a constant that shifts the sigmoid function horizontally.

These kernel functions map data to a higher-dimensional space for easier class separation. Fig. 22. shows the basic architecture of multi-class SVM.

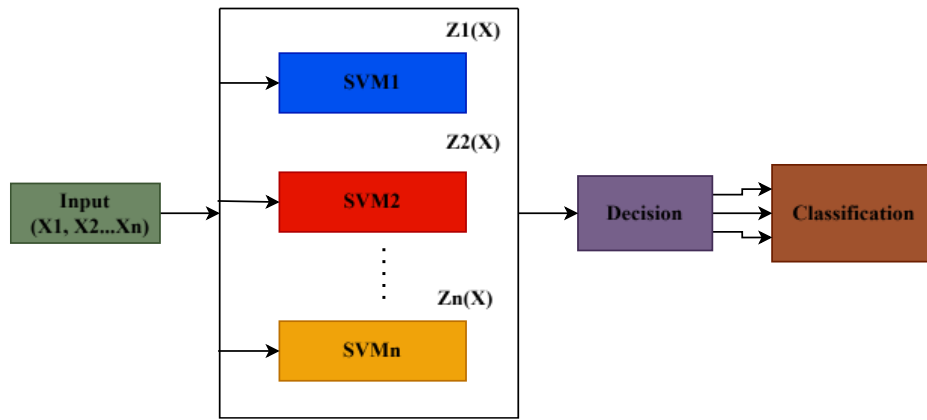


Figure 36: Architecture of Multi Class SVM

5.5.2. Deep Learning Model

For our deep learning pipeline, we utilized all five features described earlier. We employed a bi directional LSTM network for classification. Further details are provided below. Figure 23. illustrates the architecture of our deep learning pipeline.

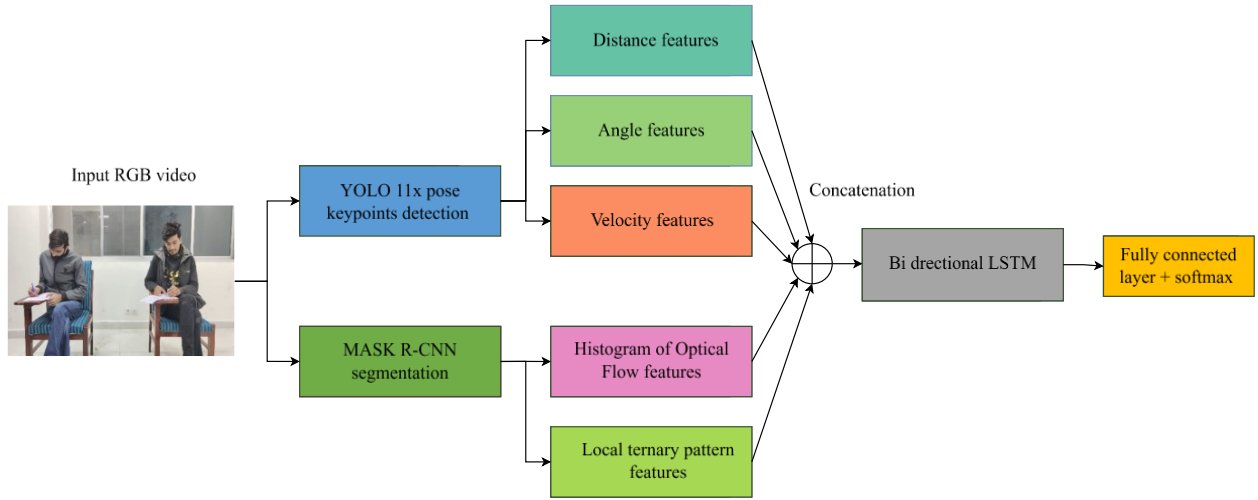


Figure 37: Architecture of Deep Learning pipeline

5.5.2.1. Bidirectional LSTM classification

LSTMs have been widely used for activity classification, primarily due to their ability to model temporal dependencies across frame sequences. Bidirectional LSTMs (BiLSTMs), which process sequences in both forward and backward directions, have shown improved performance by incorporating context from the entire sequence. We use BiLSTM for temporal modeling. Each feature is scaled individually to ensure equal contribution during classification. These scaled features are concatenated into a single vector per frame, organized into a frame-level matrix and passed to the BiLSTM, allowing it to effectively learn complex motion dynamics for classification.

At its core, LSTM uses gating mechanisms to control the flow of information. The input gate i_t controls how much new information is added to the cell state. The forget gate f_t regulates how much of the previous cell state c_{t-1} should be forgotten. The cell state update $\tilde{c}_t$ computes the candidate cell state which serves as a potential new state for the LSTM. The updated cell state c_t combines c_{t-1} and $\tilde{c}_t$, creating the new cell state for the current time step t . The output gate o_t controls how much of the cell state c_t should be output as the hidden state. Finally, the hidden state h_t is the final output of the LSTM at t . The equations for these operations are as follows:

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (13)$$

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (14)$$

$$\tilde{c}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c) \quad (15)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \quad (16)$$

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (17)$$

$$h_t = o_t \odot \tanh(c_t) \quad (18)$$

Where h_{t-1} is the previous hidden state, x_t is the input feature vector at t , σ denotes the sigmoid function, $\odot$ represents element-wise multiplication, and $\tanh$ is the hyperbolic tangent function. The terms W_i , W_f , W_c , W_o are weight matrices, and b_i , b_f , b_c , b_o are bias terms. Figure 24. shows cell structure of LSTM used in our deep learning pipeline.

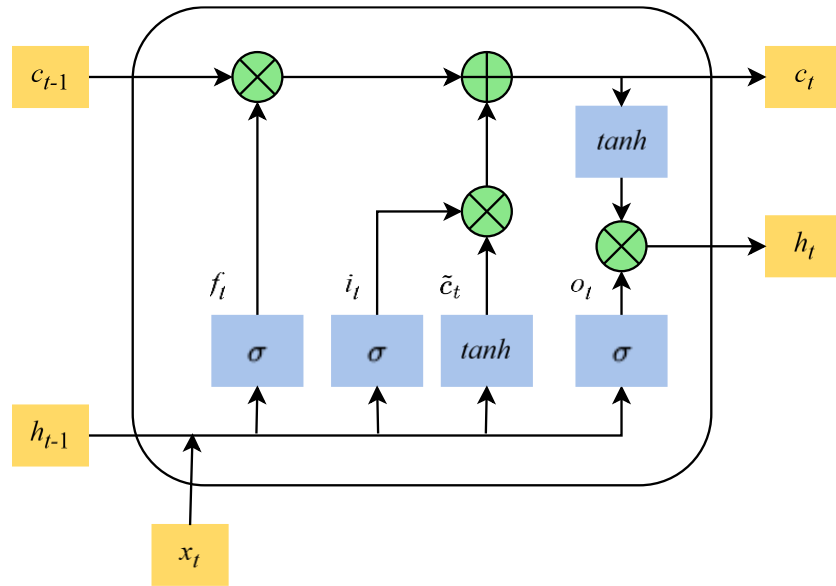


Figure 38: LSTM cell Architecture

In BiLSTM, forward and backward passes are processed separately, and their hidden states are concatenated at each time step. The final hidden state is formed by combining the forward hidden state from the last time step and the backward hidden state from the first. This representation captures the entire sequence context, making it ideal for modeling complex temporal patterns. It is then passed through a fully connected layer, which maps it to a vector representing the activity classes. This output is then processed through a softmax function to generate class probabilities. The class with the highest probability is selected as the prediction. Figure 25. shows a visualization of the architecture of BiLSTM.

In our experiments, we explored various BiLSTM parameters, including different numbers of hidden units, layers, learning rates, and learning rate schedulers to achieve the best possible accuracy.

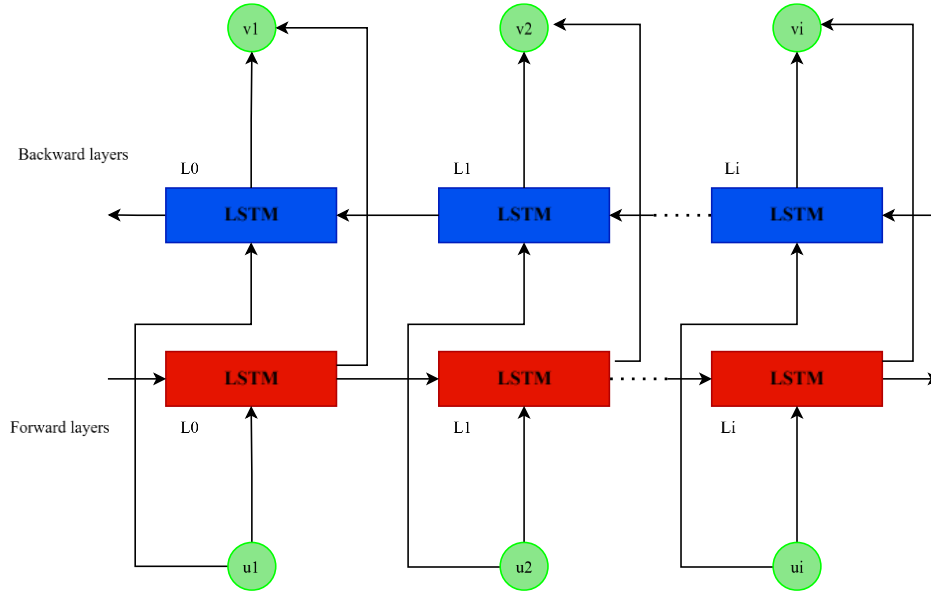


Figure 39: Architecture of BiLSTM

5.6. Experimental Results

5.6.1. Confusion matrix

To gain a deeper understanding of the classification performance, we computed the confusion matrix for both the machine learning and deep learning pipelines. These are shown below:

Table 30: Confusion matrix for machine learning pipeline

Classes	DO	PA	LO
DO	0.90	0.00	0.10
PA	0.07	0.90	0.03
LO	0.17	0.00	0.83
Mean accuracy = 87.9%			

DO=doing own work, PA=passing paper, LO=looking at other's work

Table 31: Confusion matrix for deep learning pipeline

Classes	LO	LO	PA
DO	0.90	0.00	0.10
LO	0.00	0.97	0.03
PA	0.13	0.07	0.80
Mean accuracy = 88.8%			

DO=doing own work, PA=passing paper,LO=looking at other's work

5.6.2. Precision, Recall, and F1-Score

We also calculated the precision, recall and f1-score achieved by machine and deep learning pipelines. These are shown in the following tables.

Table 32: Measures of Precision, Recall and F1 score for machine learning pipeline

Classes	Precision	Recall	F1 score
DO	0.81	0.90	0.85
PA	1.00	0.91	0.95
LO	0.86	0.83	0.84
Average	0.89	0.88	0.88

DO=doing own work, PA=passing paper,LO=looking at other's work

Table 33: Measures of Precision, Recall and F1 score for deep learning pipeline

Classes	Precision	Recall	F1 score
DO	0.87	0.91	0.89
PA	0.93	0.97	0.95
LO	0.86	0.80	0.83
Average	0.89	0.89	0.89

DO=doing own work, PA=passing paper,LO=looking at other's work

5.7. Components, Libraries, Web Services and Stubs

This section outlines the key components, libraries, web services, and stubs used in the development and deployment of the platform. The details are as follows:

5.7.1. Components

The following components are essential to the platform's functionality:

5.7.1.1. Frontend (Next.js)

The frontend is built using Next.js, providing a responsive and user-friendly interface. It includes the following pages:

- **Admin Panel:** A restricted page for administrators to manage users, view analytics, and configure platform settings.
- **Pricing Page:** Displays subscription plans, benefits, and pricing details.
- **Unsubscribed User Page:** Allows unsubscribed users to upload videos and get classification results using the default machine learning model. No storage is provided for these users.
- **Subscribed User Page:** Enables subscribed users to upload videos, select between machine learning or deep learning models, view classification results, and store the video data in MongoDB and Google Cloud Storage (GCS).
- **Video Viewing Page:** Available only to subscribed users, this page allows them to view previously uploaded videos and their classification results.
- **Login Page:** Allows users to authenticate into the platform.
- **Signup Page:** Enables new users to create an account on the platform.
- **OTP Verification Page:** Used for two-factor authentication to verify users' identity during the signup process.

5.7.1.2. Backend (Django)

The backend is powered by Django, managing server-side logic, user authentication, and database operations. Key components include:

- **User Management:** Handles user registration, login, subscription management, and OTP verification.
- **Admin Panel Functionality:** Provides admins with the ability to manage users, monitor activity, and configure platform settings.
- **Data Management:** Facilitates the storage and retrieval of video data and related metadata.

5.7.1.3. User Roles and Functionality

- **Subscribed Users:** Have access to premium features such as video uploads, selection of models (machine or deep learning), video storage in GCS, and storage of classification results in MongoDB.
- **Unsubscribed Users:** Can upload videos and receive classification results using the default machine learning model, but without storage capabilities.
- **Admin Users:** Can control subscription of users as well as remove user accounts from platform in case of misconduct.

5.7.1.4. ML and DL Pipelines

ML and DL Pipelines are a core component of the platform, enabling automatic classification of activities in uploaded videos. It consists of the following sub-components:

- **Feature extraction:** Raw video data is processed using feature extraction techniques such as Optical Flow (HOF), Velocity, and Distance features.
- **Classification:** The features are put into the classifier of the respective pipeline for classification.
- **Model Storage:** The models are stored on a AI deployment service and can be updated with new versions as required.

5.7.1.5. Libraries

The following libraries are used to support various aspects of the system:

- **Pandas:** A powerful data analysis and manipulation library used for handling structured data, particularly during preprocessing and feature extraction in the classification pipeline.
- **NumPy:** A library for numerical computations, widely used for array manipulation and mathematical operations.
- **PyTorch:** A deep learning framework used for training and implementing neural networks for activity classification.
- **scikit-learn:** A machine learning library used for implementing classical machine learning algorithms like Support Vector Machines (SVM).
- **Matplotlib:** A plotting library used to visualize data, model results, and performance metrics.

- **Next.js Libraries:** Includes libraries like react, react-dom, next/router, and others required for server-side rendering, routing, and managing the frontend components of the platform.
- **Django Libraries:** Includes core Django libraries, such as django.rest_framework for API development, django.contrib.auth for user authentication, and other essential libraries for backend functionality.

5.7.1.6. Web Services

- **Google Cloud Storage (GCS):** A cloud storage service for storing video data uploaded by subscribed users.
- **MongoDB:** A NoSQL database used to store metadata related to video uploads, classification results, and user data (e.g., subscription status).

5.7.1.7. Stubs

- **API Stub:** A mock version of the backend API used for testing the frontend's interaction with the server before full implementation.
- **Model Stub:** A placeholder for the classification models used during development, allowing testing of the pipeline's integration with mock predictions before the final model is deployed.

5.8. Deployment environment

The deployment of the platform was structured across different services to ensure scalability, reliability, and ease of management:

- **Frontend Deployment:**
The frontend application, built with Next.js, was deployed as a static service on Render. This setup allows the frontend to serve static pages efficiently with fast load times and optimized performance.
- **Backend Deployment:**
The backend application, developed using Django, was also deployed on Render. This ensures seamless API communication between the frontend and backend, managing user authentication, video processing requests, and database interactions.
- **ML and DL Pipelines Deployment:**
The machine learning (ML) and deep learning (DL) pipelines were deployed separately

on Modal, a platform designed to host and scale AI workloads. Modal allows dynamic execution of the classification models in response to user uploads without burdening the backend server.

- **Database Services:**

- **MongoDB Atlas:** Used as a cloud-hosted NoSQL database for storing user metadata, subscription details, and classification results.
- **Google Cloud Storage (GCS):** Used for storing uploaded video files from subscribed users securely in the cloud.

This deployment architecture separates concerns between user interface, server operations, model inference, and data storage, improving both system efficiency and maintainability.

5.9. Tools and Technologies

Tools Used:

- **Visual Studio Code (VS Code):** Used as the primary code editor for frontend and backend development, offering powerful extensions for Python, JavaScript, and web frameworks.
- **Postman:** Used extensively for testing and validating backend APIs during the development and deployment phases.

Technologies Used:

- **Next.js:** A React-based framework used for building the frontend application with server-side rendering and static site generation features.
- **Django:** A high-level Python web framework used for building the backend REST APIs and handling server-side logic.
- **PyTorch:** A deep learning framework used for implementing and deploying the deep learning model for activity recognition.
- **scikit-learn:** A machine learning library used for implementing and deploying classical machine learning models.
- **MongoDB Atlas:** A cloud-hosted NoSQL database service for scalable and flexible data storage.
- **Google Cloud Storage:** A cloud storage solution used to store video files securely.

- **Render:** A cloud platform used for deploying both the frontend (Next.js) and backend (Django) services.
- **Modal:** A platform for deploying, scaling, and managing AI and ML model inference services.

5.10. Version control

Version control was managed using Git and GitHub throughout the development of the platform. GitHub was used to host the project repository, providing a centralized platform for collaboration, backup, and version management.

Key version control practices included:

- **Regular Commits:** Frequent commits were made after implementing key features or making significant changes to the codebase to ensure a detailed project history.
- **Branching Strategy:** Separate branches were created for major features or updates, allowing independent development without affecting the main production branch.
- **Pull Requests and Code Reviews:** New changes were merged into the main branch only after thorough review and testing through pull requests, maintaining code stability.
- **Backup and Collaboration:** GitHub served as a remote repository ensuring that the project was securely backed up and could support collaboration if needed.

This approach to version control helped maintain a clean and organized codebase and made it easy to track changes over the course of development.

5.11. Best practices/coding standards

Best coding practices were followed to ensure that the codebase was maintainable, scalable, and easy to understand:

- **Component-Based Development:** The frontend was built using a React component-based approach within Next.js, promoting modularity and reusability of code. Each functional part of the UI (e.g., login page, video upload page) was created as a separate component to keep the code clean and organized.
- **Separation of Concerns:** Logical separation between frontend (Next.js), backend (Django), and machine learning pipelines ensured that different parts of the system had clearly defined responsibilities.

- **Consistent Coding Style:** Code formatting, naming conventions, and folder structures were kept consistent throughout the project to enhance readability and ease of debugging.
- **Environment Variables:** Sensitive information, such as API keys and database connection strings, were handled using environment variables to avoid hardcoding credentials in the source code.
- **Error Handling:** Basic error handling was implemented, especially in backend APIs and user input forms, to ensure a smoother user experience and to prevent system crashes.
- **Code Documentation and Comments:** Functions, classes, and critical logic segments were documented with comments to make the code more understandable for future maintenance or for new developers joining the project.

While the project largely focused on component-based architecture and clear separation between services, future improvements can include adding automated code formatting tools (e.g., Prettier, Black), unit testing, and stricter typing practices for even higher quality assurance.

Chapter 6

Business Plan

Chapter 6: Business Plan

This chapter focuses on the commercial viability and strategic foundation of the Intelligent Group Interactive Surveillance and Management System. It outlines the project's purpose, market positioning, and business goals. Key components include a clear business description, market strategy, competitor overview, and a breakdown of the services provided. A SWOT analysis is also included to evaluate the system's strengths and identify potential risks.

6.1 Business Description

The Intelligent Group Interactive Surveillance and Management System is a web-based AI-powered platform designed to enhance exam integrity in academic institutions. It leverages machine learning and deep learning techniques to detect and classify group cheating behavior from classroom video footage. This system reduces reliance on manual invigilation and introduces an automated, scalable, and efficient monitoring solution.

6.2 Market Analysis & Strategy

The demand for smart invigilation tools has grown due to increased concerns over academic dishonesty and the limitations of traditional surveillance methods. Target markets include schools, colleges, universities, and examination centers. The strategy involves providing an affordable, AI-powered alternative to expensive commercial proctoring tools. Initial outreach will be through academic conferences, educational platforms, and direct engagement with institutions. The system's freemium model attracts users while premium subscriptions offer cloud storage and enhanced features.

6.3 Competitive Analysis

Currently there aren't many solutions in the market giving the business a good opportunity for growth. Our system uses both ML and DL pipelines to classify complex group activities of students, making it more accurate and adaptable for classroom settings. It also supports cloud storage, allowing users to do analysis later. This makes it competitive and unique than other systems in the market.

6.4 Products/Services Description

The main product is an intelligent web-based application that accepts uploaded classroom video footage and processes it to detect cheating behaviors using trained ML and DL models. Services include:

- Role-based access for subscribed/unsubscribed users
- Admin panel for managing users and subscriptions
- Institution owner dashboard for managing faculty.
- Faculty member dashboard for uploading videos, managing video data and getting results.
- Outputs return classification result in pdf form. Premium (Faculty) users gain access to cloud storage and advanced model features, while unsubscribed users have no cloud storage and ML-based classification.

6.5 SWOT Analysis

Strengths:

- AI-powered, scalable, and cost-effective solution
- Detects contextual group activities, not just individual activities
- Web-based and accessible from any platform

Weaknesses:

- The system supports only pre-recorded video analysis and does not operate in real-time environments.
- Requires continuous model updates for different environments
- Does not perform well when there are more than 2 students performing activity in a video

Opportunities:

- Expansion into online learning platforms
- Integration with smart campus infrastructure
- Partnerships with educational ministries and boards

Threats:

- Competition from established proctoring services
- Data privacy concerns and regulatory compliance

Chapter 7

Testing & Evaluation

Chapter 7: Testing and Evaluation

This chapter validates the functionality, reliability, and performance of the video activity classification system. It covers use case testing, equivalence partitioning, boundary value analysis, and performance/stress testing to ensure the system meets requirements under various conditions. Testing focuses on:

- **Authentication** (Login/Signup/OTP).
- **Video Processing** (ML/DL pipelines).
- **User Roles** (Admin/Regular users).
- **Subscription Tiers** (Premium/Free).

7.1. Use Case Testing (Test Cases)

- Valid User Registration

Table 34: Test Case TC-1 — Successful User Registration Flow

Test Case ID	TC 01	Test Case Description	User Registration		
Created By	Areeb	Reviewed By	Awais	Version	1.0
QA tester log	Verify successful user signup a				
Tester Name	Areeb	Date Tested	1-Apr-2025	Test Case (Pass/Fail/Not Executed)	Pass
S #	Prerequisites:		S #	Test Data	
1	Access to Chrome Browser		1	Email = test@email.com Pass = Test@12345	
2.	Email not exists in MongoDB		2	Name= test Phone no = 12345678910	
Test Scenario	Verify successful user signup				
Step #	Step Details	Expected Results	Actual Results	Pass / Fail / Not executed / Suspended	
1	POST/signup with name, email, password and phone number	HTTP 200 + JWT tokens	As Expected	Pass	

- Valid Registration(Admin User)

Table 35:Test Case TC - 2 — Successful Admin Registration Flow

Test Case ID	TC_02	Test Case Description	Valid Registration (Admin User)		
Created By	Areeb	Reviewed By	Awais	Version	1.0
QA tester log	Admin can register with valid details				
Tester Name	Areeb	Date Tested	1-Apr-2025	Test Case (Pass/Fail/Not Executed)	Pass
S #	Prerequisites:		S #	Test Data	
1	Access to Chrome Browser		1	Email = admin@email.com	
2				Pass = Admin@123	
Test Scenario	Admin register with valid details				
Step #	Step Details	Expected Results	Actual Results	Pass / Fail / Not executed / Suspended	
1	POST/register with valid admin data	HTTP 200, OTP sent, admin=True in DB	As Expected	Pass	

- **Duplicate Email Registration**

Table 36: Test Case TC - 3 — Blocking Duplicate Email Registration

Test Case ID	TC 03	Test Case Description	Block Duplicate Email Sign-up		
Created By	Areeb	Reviewed By	Awais	Version	1.0
QA tester log	Block Sign-up with existing email				
Tester Name	Areeb	Date Tested	1-Apr-2025	Test Case (Pass/Fail/Not Executed)	Pass
S #	Prerequisites:		S #	Test Data	
1	Access to Chrome Browser		1	Email = test@email.com Pass = Test@12345	
2	Email Exists in MongoDB		2	Name= test Phone no = 12345678910	
Test Scenario	Test Sign-up with existing email				
Step #	Step Details	Expected Results	Actual Results		Pass / Fail / Not executed / Suspended
1	POST/signup with existing email	HTTP 409 Conflict	As Expected		Pass

- **Sign-up with Weak Password**

Table 37: Test Case TC - 4 — Password Strength Validation on Sign-Up

Test Case ID	TC_04	Test Case Description	Check Password Strength		
Created By	Areeb	Reviewed By	Awais	Version	1.0
QA tester log	Refrain Sign-up with Weak Password(min 8 char)				
Tester Name	Areeb	Date Tested	1-Apr-2025	Test Case (Pass/Fail/Not Executed)	Pass
S #	Prerequisites:		S #	Test Data	
1	Access to Chrome Browser		1	Email = test@email.com Pass = 123	
2	Email not Exists in MongoDB		2	Name= test Phone no. 12345678910	
Test Scenario	Check password(min 8 char, 1 special char, 1 upper and lowercase letter)				
Step #	Step Details	Expected Results	Actual Results		Pass / Fail / Not executed / Suspended
1	POST/signup with password = 123	HTTP 400 + Password too weak	As Expected		Pass

- **Password/Confirmation Password Mismatch**

Table 38: Test Case TC - 5 — Check Password and Confirm Password match

Test Case ID	TC_05	Test Case Description	Verify system rejects password/confirm_password mismatch during registration		
Created By	Areeb	Reviewed By	Awais	Version	1.0
QA tester log	System should return HTTP 400 when password don't match				
Tester Name	Areeb	Date Tested	1-Apr-2025	Test Case (Pass/Fail/Not Executed)	Pass
S #	Prerequisites:		S #	Test Data	
1	Access to registration endpoint(/signup)		1	Name = test user Email = test@email.com	
2	No existing user with email			Pass = Test@123 Confirm Pass = Test@456	
Test Scenario	User attempts registration with mismatched password and confirm password fields				
Step #	Step Details	Expected Results	Actual Results		Pass / Fail / Not executed / Suspended
1	Send POST request to /signup/ with mismatched passwords (see Test Data).	HTTP 400,{‘error’:‘Password and confirm password don’t match”	As Expected		Pass

- **Missing Required Fields**

Table 39: Test Case TC - 6 — Rejecting Sign-Up with Missing Required Fields

Test Case ID	TC_06	Test Case Description	Reject signup with missing fields		
Created By	Areeb	Reviewed By	Awais	Version	1.0
QA tester log	Refrain from signing up with missing fields				
Tester Name	Areeb	Date Tested	1-Apr-2025	Test Case (Pass/Fail/Not Executed)	Pass
S #	Prerequisites:		S #	Test Data	
1	Access to Chrome Browser		1	Email = test@email.com Pass = missing	
2	Visit Sign-up page		2	Name= test Phone no. 12345678910	
Test Scenario	Test Sign-up with missing fields				
Step #	Step Details	Expected Results	Actual Results		Pass / Fail / Not executed / Suspended
1	POST/signup without password	HTTP 400 + “Field Required”	As Expected		Pass

- **Verify OTP**

Table 40:Test Case TC - 7 — OTP Verification after Successful Sign-Up

Test Case ID	TC 07	Test Case Description	Verify OTP		
Created By	Areeb	Reviewed By	Awais	Version	1.0
QA tester log	Verify successful OTP validation after signup				
Tester Name	Areeb	Date Tested	1-Apr-2025	Test Case (Pass/Fail/Not Executed)	Pass
S #	Prerequisites:		S #	Test Data	
1	Access to Chrome Browser		1	email = test@email.com otp = 123456	
2	User has signed up and received OTP via email			Pass = Test@12345	
Test Scenario	Verify successful OTP validation				
Step #	Step Details	Expected Results	Actual Results		Pass / Fail / Not executed / Suspended
1	POST/otp with test data	HTTP 200 + JWT tokens	As Expected		Pass
2	Check MongoDB for blocked =false	blocked = false	As Expected		Pass

- **Resend OTP**

Table 41: Test Case TC - 8 — Resend OTP Functionality for Unverified Users

Test Case ID	TC_08	Test Case Description	Resend OTP to email		
Created By	Areeb	Reviewed By	Awais	Version	1.0
QA tester log	Ensure OTP resend				
Tester Name	Awais	Date Tested	1-Apr-2025	Test Case (Pass/Fail/Not Executed)	Pass
S #	Prerequisites:		S #	Test Data	
1	Access to Chrome Browser		1	email = test@email.com otp = 123456	
2	User exists but unverified			Pass = Test@12345	
Test Scenario	Validate OTP resend				
Step #	Step Details	Expected Results	Actual Results		Pass / Fail / Not executed / Suspended
1	POST/resend-otp with email	HTTP 202 + new OTP	As Expected		Pass

- Expired OTP

Table 42: Test Case TC - 9 — Resend OTP Functionality for Unverified Users

Test Case ID	TC_09	Test Case Description	Verify system rejects expired OTP during verification		
Created By	Areeb	Reviewed By	Awais	Version	1.0
QA tester log	System should return HTTP 410 when expired OTP is submitted				
Tester Name	Areeb	Date Tested	1-Apr-2025	Test Case (Pass/Fail/Not Executed)	Pass
S #	Prerequisites:		S #	Test Data	
1	User with test@email.com exist in DB		1	Email =test@email.com Pass = Test@12345	
2	Valid OTP generated and timestamp recorded in DB			OTP = 123456(valid but expired)	
Test Scenario	User submits an OTP that has expired (6 minutes old).				
Step #	Step Details	Expected Results	Actual Results		Pass / Fail / Not executed / Suspended
1	Send POST request to /verify-otp/ with expired otp	HTTP 410,{"error": "OTP has expired"}	As Expected		Pass

- **System Blocks OTP Verification After 3 Failed Attempts**

Table 43: Test Case TC - 10 — Block OTP verification after 3 failed attempts

Test Case ID	TC_10	Test Case Description	Verify system blocks OTP verification after 3 failed attempts		
Created By	Areeb	Reviewed By	Awais	Version	1.0
QA tester log	System should lock OTP verification after 3 invalid attempts				
Tester Name	Areeb	Date Tested	1-Apr-2025	Test Case (Pass/Fail/Not Executed)	Pass
S #	Prerequisites:		S #	Test Data	
1	User with test@email.com exists and blocked=True		1	email = test@email.com otp = 123456 Pass = Test@12345 Invalid OTP 1: 111111	
2	Valid OTP generated and stored in DB			Invalid OTP 2; 222222 Invalid OTP 3: 333333	
Test Scenario	User fails OTP verification 3 times consecutively.				
Step #	Step Details	Expected Results	Actual Results		Pass / Fail / Not executed / Suspended
1	Submit 1st invalid OTP via POST /verify-otp/	HTTP 400 + "Attempts remaining: 2"	As Expected		Pass
2	Submit 2nd invalid OTP	HTTP 400 + "Attempts remaining: 1"	As Expected		Pass
3	Submit 3rd invalid OTP	HTTP 429 + "Maximum attempts reached"	As Expected		Pass

• Phone Number Validation

Table 44: Test Case TC - 11 — Phone Number Format Validation on Sign-Up

Test Case ID	TC_011	Test Case Description	Checks the phone number		
Created By	Areeb	Reviewed By	Awais	Version	1.0
QA tester log	No Sign up with invalid number				
Tester Name	Areeb	Date Tested	1-Apr-2025	Test Case (Pass/Fail/Not Executed)	Pass
S #	Prerequisites:		S #	Test Data	
1	Access to Chrome Browser		1	email = test@email.com Pass = Test@12345	
2	Acess the Sign Up page			otp = 123456 Phone no = 12345	
Test Scenario	Phone number is valid or invalid				
Step #	Step Details	Expected Results	Actual Results		Pass / Fail / Not executed / Suspended
1	POST/sign-up with phone=invalid	HTTP 400 with invalid phone	As Expected		Pass

- **Login with Valid Credentials**

Table 45: Test Case TC - 12 — Login with Valid Credentials

Test Case ID	TC_12	Test Case Description	Test login with valid credentials		
Created By	Areeb	Reviewed By	Awais	Version	1.0
QA tester log	Verified JWT token generation				
Tester Name	Areeb	Date Tested	1-Apr-2025	Test Case (Pass/Fail/Not Executed)	Pass
S #	Prerequisites:		S #	Test Data	
1	Access to Chrome Browser		1	Email = test@email.com	
2	User exists in MongoDB			Pass = Test@12345	
Test Scenario	Verify successful login with valid credentials.				
Step #	Step Details	Expected Results	Actual Results	Pass / Fail / Not executed / Suspended	
1	POST /login with correct email and password	HTTP 200 + JWT tokens	As Expected	Pass	

- **Invalid Login**

Table 46:Test Case TC - 13 — Reject Login with Invalid Credentials

Test Case ID	TC_13	Test Case Description	Test login with invalid credentials		
Created By	Areeb	Reviewed By	Awais	Version	1.0
QA tester log	Reject login with wrong password				
Tester Name	Areeb	Date Tested	1-Apr-2025	Test Case (Pass/Fail/Not Executed)	Pass
S #	Prerequisites:		S #	Test Data	
1	Access to Chrome Browser		1	Email = test@email.com	
2	User exists in MongoDB			Pass = test@12345	
Test Scenario	Test login with invalid credentials.				
Step #	Step Details	Expected Results	Actual Results	Pass / Fail / Not executed / Suspended	
1	POST /login with invalid credentials	HTTP 401 Unauthorized	As Expected	Pass	

- **System Accepts Only Video File**

Table 47: Test Case TC - 14 — Reject non video file

Test Case ID	TC_14	Test Case Description	Verify system accepts only video file uploads (MP4)		
Created By	Areeb	Reviewed By	Awais	Version	1.0
QA tester log	System should reject non-video files during upload				
Tester Name	Areeb	Date Tested	1-Apr-2025	Test Case (Pass/Fail/Not Executed)	Pass
S #	Prerequisites:		S #	Test Data	
1	Valid faculty user logged in (JWT token available)		1	Valid File : video.mp4 Invalid File: invalid.txt, invalid.pdf	
2	Test files available: valid.mp4,invalid.txt, invalid.pdf				
Test Scenario	System validates file types during video upload.				
Step #	Step Details	Expected Results	Actual Results		Pass / Fail / Not executed / Suspended
1	POST /upload-video/ with valid.mp4	HTTP 200 + classification Results	As Expected		Pass
2	POST /upload-video/ with invalid.txt	HTTP 400 + "Invalid video format"	As Expected		Pass

- **Verify system handles corrupted video upload**

Table 48: Test Case TC - 15 — Handle Corrupted video upload

Test Case ID	TC_15	Test Case Description	Verify system handles corrupted video upload		
Created By	Areeb	Reviewed By	Awais	Version	1.0
QA tester log	System should reject corrupted video files with proper error				
Tester Name	Areeb	Date Tested	1-Apr-2025	Test Case (Pass/Fail/Not Executed)	Pass
S #	Prerequisites:		S #	Test Data	
1	Valid faculty user logged in (JWT token available)		1	Email =test@email.com	
2	Corrupted video file available (corrupt.mp4)			Video File= corrupt.mp4(0 Byte or Header Damage) Model Type = DL	
Test Scenario	User uploads a corrupted video file.				
Step #	Step Details	Expected Results	Actual Results		Pass / Fail / Not executed / Suspended
1	POST /upload-video/ with corrupted file	HTTP 500	As Expected		Pass

- **Verify concurrent video uploads**

Table 49:Test Case TC - 16 — Verify concurrent video uploads

Test Case ID	TC_16	Test Case Description	Verify concurrent video uploads		
Created By	Areeb	Reviewed By	Awais	Version	1.0
QA tester log	System should handle 5+ simultaneous uploads				
Tester Name	Areeb	Date Tested	1-Apr-2025	Test Case (Pass/Fail/Not Executed)	Pass
S #	Prerequisites:		S #	Test Data	
1	5 faculty users logged in (unique JWT tokens)		1	Video-file: video1.mp4 to video5.mp4	
2	5 unique test videos (video1.mp4 to video5.mp4)				
Test Scenario	Handle 5 video uploads concurrently				
Step #	Step Details	Expected Results	Actual Results		Pass / Fail / Not executed / Suspende d
1	Simultaneously POST to /upload-video/ from 5 users	All HTTP 200	As Expected		Pass

- **Common User Upload Video without Metadata**

Table 50:Test Case TC - 17 — Verify common user can upload video without metadata

Test Case ID	TC_17	Test Case Description	Verify common user can upload video without metadata		
Created By	Areeb	Reviewed By	Awais	Version	1.0

QA tester log	Common user's video should process but not store metadata				
Tester Name	Areeb	Date Tested	1-Apr-2025	Test Case (Pass/Fail/Not Executed)	Pass
S #	Prerequisites:		S #	Test Data	
1	Common user logged in (valid JWT, subscription=False)		1	Email = test@email.com Pass = Test@12345 Video-file: mp4	
2	Valid MP4 video file (user_video.mp4)				
Test Scenario	Common user logged in (valid JWT, subscription=False)				
Step #	Step Details	Expected Results	Actual Results	Pass / Fail / Not executed / Suspende d	
1	POST /upload-video/ with video file (no metadata)	HTTP 200 + classification results	As Expected	Pass	

- **Faculty Member Uploads Video with Metadata**

Table 51: Test Case TC - 18 — Verify Faculty member uploads video with metadata

Test Case ID	TC_18	Test Case Description	Faculty member uploads video with metadata		
Created By	Areeb	Reviewed By	Awais	Version	1.0
QA tester log	Faculty member uploads video with metadata				
Tester Name	Areeb	Date Tested	1-Apr-2025	Test Case (Pass/Fail/Not Executed)	Pass
S #	Prerequisites:		S #	Test Data	
1	Access to Chrome Browser		1	Video_file = midterm_exam.mp4	
2	Faculty member logged in			Subject = Physics Exam_type = Mid Term	
Test Scenario	Faculty Members upload video with meta data				
Step #	Step Details	Expected Results	Actual Results		Pass / Fail / Not executed / Suspended
1	POST /upload-video/	HTTP 200 + classification results	As Expected		Pass
2	Verify metadata stored in MongoDB	exam_type="Midterm" recorded	As Expected		Pass

- **Institutional admin creates faculty member**

Table 52: Test Case TC 19 — Institutional admin creates faculty member

Test Case ID	TC_19	Test Case Description	Institutional admin creates faculty member		
Created By	Awais	Reviewed By	Areeb	Version	1.0
QA tester log	Institutional admin assigns faculty member dashboards				
Tester Name	Awais	Date Tested	1-Apr-2025	Test Case (Pass/Fail/Not Executed)	Pass
S #	Prerequisites:		S #	Test Data	
1	Institutional admin Login		1	Name = Faculty Member	
2	Institutional admin has premium subscription			Email = faculty@email.com Pass = admin@123	
Test Scenario	Institutional admin can create faculty members				
Step #	Step Details	Expected Results	Actual Results		Pass / Fail / Not executed / Suspended
1	POST /create-faculty/	HTTP 202 + OTP sent	As Expected		Pass

- **Delete Video (Faculty-Premium)**

Table 53:Test Case TC - 20 — Faculty can delete stored video

Test Case ID	TC_20	Test Case Description	Faculty can deletes stored video		
Created By	Awais	Reviewed By	Areeb	Version	1.0
QA tester log	Delete Video by Premium User				
Tester Name	Awais	Date Tested	1-Apr-2025	Test Case (Pass/Fail/Not Executed)	Pass
S #	Prerequisites:		S #	Test Data	
1	Faculty Member logged In		1	Email = faculty@email.com	
2	Video exists in MongoDB/GCS			Pass = Test@12345	
Test Scenario	Check if faculty member can delete video uploaded by him				
Step #	Step Details	Expected Results	Actual Results		Pass / Fail / Not executed / Suspended
1	DELETE/delete-video/video123.mp4	HTTP 200 + video removed from GCS/MongoDB	As Expected		Pass

- **Admin Access Control**

Table 54: Test Case TC- 21 — Block non-admins from user management

Test Case ID	TC_21	Test Case Description	Block non-admins from user management		
Created By	Awais	Reviewed By	Areeb	Version	1.0
QA tester log	Only Admin can manage users				
Tester Name	Awais	Date Tested	1-Apr-2025	Test Case (Pass/Fail/Not Executed)	Pass
S #	Prerequisites:		S #	Test Data	
1	Access to Chrome Browser		1	Email = test@email.com Pass = Test@12345	
2	Regular User logged-in				
Test Scenario	Check whether the non-admin user can delete the other user				
Step #	Step Details	Expected Results	Actual Results	Pass / Fail / Not executed / Suspended	
1	DELETE/delete-user/any@test.com	HTTP 403 forbidden	As Expected	Pass	

- **Toggle Subscription by System Admin**

Table 55: Test Case TC 22 — Admin Toggles Subscription Status

Test Case ID	TC_22	Test Case Description	Admin can Change Subscription Plan		
Created By	Areeb	Reviewed By	Awais	Version	1.0
QA tester log	System Admin can toggle the subscription of Institutional admin				
Tester Name	Areeb	Date Tested	1-Apr-2025	Test Case (Pass/Fail/Not Executed)	Pass
S #	Prerequisites:		S #	Test Data	
1	Non-premium user exists		1	Email = admin@email.com	
2	Admin logged in			Pass = Test@12345	
Test Scenario	Admin toggling the Subscription of Institution				
Step #	Step Details	Expected Results	Actual Results	Pass / Fail / Not executed / Suspended	
1	POST/update-subscription with subscription = true	HTTP 200 + subscription:true in mongoDB	As Expected	Pass	

- Admin deletes the user

Table 56: Test Case TC 23 — Admin Deletes a User Account

Test Case ID	TC_23	Test Case Description	Admin can delete the user		
Created By	Awais	Reviewed By	Areeb	Version	1.0
QA tester log	Admin can delete any user				
Tester Name	Awais	Date Tested	1-Apr-2025	Test Case (Pass/Fail/Not Executed)	Pass
S #	Prerequisites:		S #	Test Data	
1	Access to Chrome Browser		1	Email = admin@email.com	
2	Admin logged in			Pass = Test@12345	
Test Scenario	Admin delete the user in the list				
Step #	Step Details	Expected Results	Actual Results		Pass / Fail / Not executed / Suspended
1	DELETE/delete-user/any@test.com	HTTP 200	As Expected		Pass

- **Institutional admin views faculty list**

Table 57: Test Case TC 24 — Institutional admin views faculty list

Test Case ID	TC_24	Test Case Description	Institutional admin views faculty list		
Created By	Awais	Reviewed By	Areeb	Version	1.0
QA tester log	Institutional admin can views faculty list				
Tester Name	Awais	Date Tested	1-Apr-2025	Test Case (Pass/Fail/Not Executed)	Pass
S #	Prerequisites:		S #	Test Data	
1	Access to Chrome Browser		1	Email = admin@email.com	
2	Institutional Admin logged in			Pass = admin@123	
Test Scenario	Institutional admin views faculty list				
Step #	Step Details	Expected Results	Actual Results		Pass / Fail / Not executed / Suspended
1	GET /get-faculty-members/	HTTP 200 + faculty array	As Expected		Pass
2	Verify seat count accurate	remaining_seats ≤ 10	As Expected		Pass

- **Generate PDF Report for Video**

Table 58: Test Case TC - 25 — Generate PDF Report for Video

Test Case ID	TC_25	Test Case Description	Generate PDF Report for Video		
Created By	Awais	Reviewed By	Areeb	Version	1.0
QA tester log	PDF report is generated for video after classification				
Tester Name	Awais	Date Tested	1-Apr-2025	Test Case (Pass/Fail/Not Executed)	Pass
S #	Prerequisites:		S #	Test Data	
1	Access to UI page		1	Video_File =cheating.mp4	
2	Video exists in system				
Test Scenario	PDF file generated or not				
Step #	Step Details	Expected Results	Actual Results		Pass / Fail / Not executed / Suspended
1	GET /video/{id}/report	PDF file download	As Expected		Pass

- **Faculty member views assigned subjects**

Table 59: Test Case TC 26 — Faculty member views assigned subjects

Test Case ID	TC_26	Test Case Description	Faculty member views assigned subjects		
Created By	Awais	Reviewed By	Areeb	Version	1.0
QA tester log	Faculty member can view the assign subjects				
Tester Name	Awais	Date Tested	1-Apr-2025	Test Case (Pass/Fail/Not Executed)	Pass
S #	Prerequisites:		S #	Test Data	
1	Access to Chrome Browser		1	Email = faculty@email.com	
2	Faculty Member logged in			Pass = Faculty@123	
Test Scenario	Faculty Member can view the assign subjects				
Step #	Step Details	Expected Results	Actual Results		Pass / Fail / Not executed / Suspended
1	GET /get-faculty-subjects/	HTTP 200 + subject list	As Expected		Pass
2	Verify timings included	Array of HH:MM formats	As Expected		Pass

- **Admin views analytics**

Table 60: Test Case TC - 27 — Admin views all analytics

Test Case ID	TC_27	Test Case Description	Admin views important analytics		
Created By	Awais	Reviewed By	Areeb	Version	1.0
QA tester log	Admin can view all the videos				
Tester Name	Awais	Date Tested	1-Apr-2025	Test Case (Pass/Fail/Not Executed)	Pass
S #	Prerequisites:		S #	Test Data	
1	Access to Chrome Browser		1	Email = admin@email.com	
2	Admin logged in			Pass = admin@123	
Test Scenario	Admin can view analytics like user growth, no of subscribed user and no of times models were used by users across different time periods				
Step #	Step Details	Expected Results	Actual Results		Pass / Fail / Not executed / Suspended
1	GET /get-all-users/	HTTP 200 + video list	As Expected		Pass
2	Verify all required data is included in response	All user data and model data in response	As Expected		Pass

- Institutional Admin Assign subjects to Faculty Member**

Table 61: Test Case TC - 28 — Institutional Admin Assign subjects to Faculty Member

Test Case ID	TC_28	Test Case Description	Institutional Admin Assign subjects to Faculty Member		
Created By	Awais	Reviewed By	Areeb	Version	1.0
QA tester log	Assign Subject to Faculty Member				
Tester Name	Awais	Date Tested	1-Apr-2025	Test Case (Pass/Fail/Not Executed)	Pass
S #	Prerequisites:		S #	Test Data	
1	Access to Chrome Browser		1	Email = Faculty@email.com	
2	Admin logged in			Pass = admin@123 Subject = CS	
Test Scenario	Institutional Admin assign the subject to Faculty Member				
Step #	Step Details	Expected Results	Actual Results		Pass / Fail / Not executed / Suspended
1	POST /assign-subject/	HTTP 200 + subject added	As Expected		Pass
2	Verify faculty’s timetable updated	faculty_member_subject array in DB	As Expected		Pass

- **Video Upload (Faculty-Premium)**

Table 62: Test Case TC - 29 — Video Upload by Faculty Member

Test Case ID	TC_29	Test Case Description	Video Upload(Faculty-Premium)		
Created By	Awais	Reviewed By	Areeb	Version	1.0
QA tester log	Verify premium user can upload video for DL classification				
Tester Name	Awais	Date Tested	1-Apr-2025	Test Case (Pass/Fail/Not Executed)	Pass
S #	Prerequisites:		S #	Test Data	
1	Access to Chrome Browser		1	Video_file=”cheating.mp4”	
2	User is logged in with subscription=true			Modal_type= deep learning model	
Test Scenario	Verify premium user can upload video for DL classification				
Step #	Step Details	Expected Results	Actual Results		Pass / Fail / Not executed / Suspended
1	POST/upload-video with video and modal type Verify GCS upload and MongoDB entry	HTTP 200 OK	As Expected		Pass

- **Video Upload (Common User)**

Table 63: Test Case TC - 30 — Video upload by Common User

Test Case ID	TC 30	Test Case Description	Video Upload(Non Premium User)		
Created By	Awais	Reviewed By	Areeb	Version	1.0
QA tester log	Verify premium user can upload video for ML classification				
Tester Name	Awais	Date Tested	1-Apr-2025	Test Case (Pass/Fail/Not Executed)	Pass
S #	Prerequisites:		S #	Test Data	
1	Access to Chrome Browser		1	Video_file="cheating.mp4"	
2	User is logged in with subscription=false			Modal_type= machine learning model	
Test Scenario	Verify premium user can upload video for ML classification				
Step #	Step Details	Expected Results	Actual Results		Pass / Fail / Not executed / Suspended
1	POST/upload-video with video and modal type Verify video is deleted from GCS after processing.	HTTP 200 OK	As Expected		Pass

7.2. Equivalence partitioning

Equivalence Partitioning is used to reduce test cases by dividing input into valid and invalid sets

Table 64: Equivalence Partitioning for Input Validation

Input Type	Valid Partition	Invalid Partition
Email Format	user@example.com	user@,example.com,user@.com
Password	8-20 char, mix char	< 8 char
Video Type	mp4, .MOV	.txt, .exe
Model Type	ML”, “DL”	“invalid”, “123”
Verify OTP	otp: 123456 (6 digits)	otp: "12A45B"

7.3. Boundary value analysis

Table 65: Boundary Value Analysis for Input Limits

Input	Lower Boundary	Upper Boundary
Video Size (MB)	1 MB	50 MB
Password Length	8	20
Video Duration	5 s	1m

7.4. Data flow testing

Data flow testing ensures that variables are correctly defined, used, and updated across the system components. This type of testing was applied to critical modules, particularly the video processing pipelines (ML and DL), where data (video files) flows from the frontend upload component, through backend APIs, into cloud/local storage, and finally to the AI classification model.

Key test scenarios included:

- Verifying video file format validation at upload.
- Ensuring correct routing of video files to ML or DL model based on user subscription.
- Confirming proper database entries after classification and handling of data states (e.g., video uploaded, video processed, classification returned).
- Detecting anomalies in state transitions such as missing data (e.g., broken video link, null classification result).

7.4.1. Detailed Data Flow Analysis

- **Video Upload & Classification Paths**

User → Frontend (Video File) → Django (Request) → GCS (Storage) → Modal (DL/ML Processing) → MongoDB (Storage if subscribed) → Frontend (Result)

- **Test Cases:**

- **Valid File:** MP4 video → Success (200) with GCS URL.
- **Invalid File:** TXT file → Error (400) before GCS upload.

- **OTP Verification Path**

User → Frontend (OTP Input) → Django → MongoDB (Verify OTP) → Unblock User

- **Test Cases:**

- **Valid OTP:** 123456 → User unblocked (200).
- **Expired OTP:** OTP older than 5 mins → Error (410)

- **Model Classification Path**

GCS (Video URL) → Modal (Feature Extraction) → Stored model (DL/ML) → Django (Result) → Frontend

- **Test Cases:**

- **DL Model:** Ensures appropriate features are extracted and model weights are loaded properly during inference.
- **ML Model:** Ensures appropriate features are extracted and model weights are loaded properly during inference.

7.5. Unit testing

Each module/component was tested individually to validate its functionality.

- **Frontend Components:** Video upload, user registration/login, model selection UI.
- **Backend Endpoints:** /signup, /login, /upload-video.
- **ML/DL Processing:** Individual testing with mock inputs (videos with labeled behaviors).
- **Admin Panel:** Subscription toggle and user audit logs.

7.6. Integration testing

Integration testing ensured seamless interaction between system modules. Following are the tests done:

- **Frontend ↔ Backend:** Validated token-based auth, video upload, and result rendering.
- **Backend ↔ Cloud Services:** Checked GCS uploads for DL models and MongoDB for metadata storage.
- **ML/DL Model ↔ API Layer:** Verified classification return, error handling on processing failure.

Testing Techniques Used:

- REST API mocking with Postman
- End-to-end scenario walkthroughs (Signup → Upload → Result)

Table 66: Integration Testing Results

Modules Integrated	Status
Backend + GCS	Passed
Backend + MongoDB	Passed
Frontend + Backend	Passed
Backend + ML/DL pipelines	Passed

7.7. Performance Testing

Tested system response times and throughput under typical load. Following are the tests done:

Table 67: Performance testing results

Scenario	Expected Time	Actual Time	Status
Video upload and ML processing	< 8 sec	~6 sec	Passed
Video upload with DL processing	< 15 sec	~12 sec	Passed
GCS + DB write operation	< 2 sec	~1.5 sec	Passed

7.8. Stress Testing

Test 1: Concurrent User Logins

Scenario: 50 users logging in simultaneously.

Steps:

- Simulate 50 POST requests to /login with valid credentials.
- Measure response times and success rates.

Expected Result:

- All requests complete within 3 seconds.
- 100% success rate (HTTP 200).

Test 2: Mixed Video Uploads (ML + DL)

Scenario: 20 users uploading videos (10 ML, 10 DL).

Steps:

- 10 users upload 30-sec MP4 files with model_type=ML.
- 10 users upload 30-sec MP4 files with model_type=DL.

Expected Result:

- ML processing completes within 10 sec/video.
- DL processing completes within 20 sec/video.
- No GCS upload failures

Test 3: Rapid OTP Requests

Scenario: 10 OTP requests in 1 minute.

Steps:

- Trigger /resend-otp for 10 test emails in quick succession.

Expected Result:

- All emails receive OTPs within 1 minute.
- No SMTP timeouts or MongoDB write failures

Chapter 8

Conclusion & Future Enhancements

Chapter 8: Conclusion & Future Enhancements

8.1. Achievements and Improvements

The development of the Intelligent Group Interactive Surveillance and Management System successfully met its primary objectives. The system can analyze pre-recorded classroom videos and classify various group cheating behaviors using machine learning and deep learning models. A functional web application was created using NextJS and Django, offering role-based access, secure login, and integration with cloud storage. Significant improvements were made in model accuracy and user interface design compared to initial prototypes.

8.2. Critical Review

While the proposed system effectively identifies cheating behaviors from recorded videos, real-time monitoring was not achieved due to technical and computational limitations. Some challenges were encountered in ensuring consistent model performance across varying classroom environments and camera qualities. Another limitation is its inability to classify activities in videos featuring more than two students interacting in a classroom. The primary challenge arises from the difficulty of segmentation and keypoint detection, as well as human tracking, when multiple students are involved. In such cases, students may block each other's view in the camera, leading to poorly segmented silhouettes, incorrect keypoint coordinates, and human mistracking, which negatively impacts classification accuracy. This issue could potentially be addressed by employing more advanced segmentation and keypoint detection models, and this will be the focus of our future research. Additionally, system deployment and testing were limited to a simulated environment, which may not fully reflect real-world classroom complexities. Also, the payment approach currently used is not reliable for real word deployment scenarios. This needs to be fixed with a proper payment provider like Stripe or PayPal.

8.3. Lessons Learnt

Throughout the project, valuable lessons were learned in the areas of AI model development, web integration, and user experience design. The importance of dataset diversity and model generalization was evident during testing phases. Collaborating in a team setting also highlighted the need for effective communication, time management, and version control. The experience

strengthened skills in full-stack development and exposed the challenges of deploying AI systems in real-world scenarios.

8.4. Future Enhancements/Recommendations

To further enhance the system's functionality and effectiveness, the following improvements are recommended:

- Incorporate real-time video analysis for live invigilation scenarios.
- Improve model training with more diverse and larger datasets from various classroom settings.
- Add mobile application support for easier access and monitoring.
- Add more activities like “eating food”, “shouting”, “fighting” etc.
- Ensure compliance with data privacy laws and introduce detailed logs for audit purposes.

Appendices

Appendix A: Information / Promotional Material

A.1. Standee (if any)



**Intelligent Group
Interactive
Surveillance and
Management
System**

Muhammad Hamdan
Azhar
REG ID: 210966

Muhammad Areeb
Shahid
REG ID: 211958

Muhammad Awaiz
REG ID: 200858

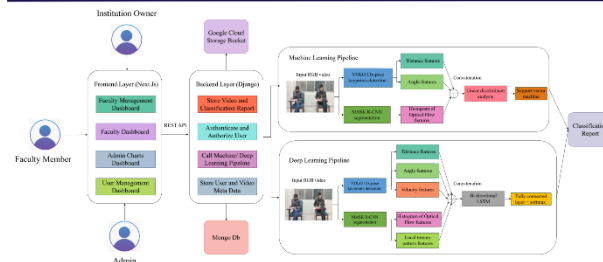
Supervisor:
Dr. Ahmad Jalal



Objective

To develop an automated surveillance system for educational institutions where institute owners can assign subjects to faculty members, who then monitor student activities during exams using computer vision. The system supports user management via an admin panel. Two classification pipelines—Machine Learning and Deep Learning—are used to detect three types of student behavior in video footage, achieving accuracies of 87.9% and 88.8% respectively. This solution aims to streamline and enhance student activity monitoring during examinations.

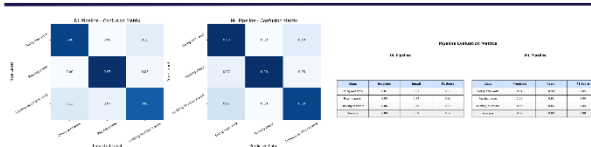
Architecture



Screens and Datasets

The screenshots show the **ExamGuard** application interface. The top section features a **Dashboard** with a **Use our AI models to detect cheating on exams and store videos on the cloud for analysis** message. Below this is a **User Growth Over Time** line chart. The **ExamGuard** section displays a **Dashboard** with a **Video Upload** form and a **Video Upload** button. The **ExamGuard** section also displays a **Video Upload** form and a **Video Upload** button. The **ExamGuard** section also displays a **Video Upload** form and a **Video Upload** button.

Results



Tools and Technologies



Department of
Computer Science

Faculty of Computing and
Artificial Intelligence

Air University
Islamabad

Reference and Bibliography

- [1] Hussein, F., Al-Ahmad, A., El-Salhi, S., Alshdaifat, E. A., & Al-Hami, M. T., "Advances in contextual action recognition: Automatic cheating detection using machine learning techniques," *Data*, 7(9), 122, 2022.
- [2] Wijaya, P. S., "Cheating Detection Through CCTV using YOLOv7,"
- [3] "Automatic recognition of human interaction via hybrid descriptors and maximum entropy Markov model using depth sensors," *Entropy*, 22(8), 817, 2020.
- [4] Gupta, S., Vishwakarma, D. K., & Puri, N. K., "A human activity recognition framework in videos using segmented human subject focus," *The Visual Computer*, 40(10), 6983-6999, 2024.
- [5] Cao, Z., Hidalgo, G., Simon, T., Wei, S. E., & Sheikh, Y., "Openpose: Realtime multi-person 2D pose estimation using part affinity fields," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 43(1), 172-186, 2019.
- [6] Perez, M., Liu, J., & Kot, A. C., "Interaction relational network for mutual action recognition," *IEEE Transactions on Multimedia*, 24, 366-376, 2021.
- [7] Li, Z., Li, Y., Tang, L., Zhang, T., & Su, J., "Two-person graph convolutional network for skeleton-based human interaction recognition," *IEEE Transactions on Circuits and Systems for Video Technology*, 33(7), 3333-3342, 2022.
- [8] He, K., Gkioxari, G., Dollár, P., & Girshick, R., "Mask R-CNN," *Proceedings of the IEEE International Conference on Computer Vision*, pp. 2961-2969, 2017.
- [9] Girshick, R., "Fast R-CNN," *Proceedings of the IEEE International Conference on Computer Vision*, pp. 1440-1448, 2015.
- [10] Jocher, G., & Qiu, J., "Ultralytics YOLO11," *Ultralytics YOLO11*, version 11.0.0, 2024, <https://github.com/ultralytics/ultralytics>, AGPL-3.0.
- [11] Farnebäck, G., "Two-frame motion estimation based on polynomial expansion," *Image Anal.: Proc. 13th Scand. Conf. SCIA 2003*, Halmstad, Sweden, Jun. 29-Jul. 2, 2003, pp. 363-370.
- [12] Fisher, R. A., "The use of multiple measurements in taxonomic problems," *Ann. Eugenics*, 7(2), 179-188, 1936.